Low-Earth Orbit Satellite Intrusion Detection System Using Deep Neural Networks

Max Ferstenberg

BSc (Hons.) Computer Science
Honours Dissertation

Supervised by Professor Oleksandr Letychevskyi

Heriot-Watt University

School of Mathematical and Computer Sciences

September 2024

ABSTRACT

In recent years, low-Earth orbit (LEO) satellites have become a cornerstone of modern communication, offering low-latency data transfer and globally scalable topologies. With this growth, however, comes an increased risk and engagement from malicious actors within the operational environment, compounded by the inherent computational limitations of the system. Contemporary IDS (Intrusion Detection Systems) have proven insufficient to address these challenges, with the detection of novel attack vectors becoming increasingly difficult, leading to overcompensation when creating models, which in turn exacerbates the persistent issue of high false positive rates in attack detection.

This paper proposes an RNN (Recurrent Neural Network) based IDS, with the novelty of including a new method of deep analysis of packets for neural classifiers, implemented successfully for the first time. The proposed IDS utilizes an augmented input vector of features extracted using GDB: the GNU Project Debugger, to describe the internal state of processes and hardware, thereby detecting deep patterns in packet sequences for improved traffic classification in LEO satellite networks. This paper uses publicly available data collected from SpaceX's Starlink LEO satellites, whose purpose is to provide internet services, along with attack vector data from the Canadian Institute of Cyber Security and the University of New South Wales. The proposed methodology describes a restricted launch in an isolated environment to perform feature extraction of the internal state of hardware and processes as traffic is processed so that we can train the proposed RNN on low-level complex relations between features to create an IDS that is more adaptable to novel attacks and starts to mitigate some of the issues surrounding contemporary IDS.

DECLARATION

I, Max Ferstenberg, confirm that this work submitted for assessment is my own and is expressed in my own words. Any uses made within it of the works of other authors in any form (e.g., ideas, equations, figures, text, tables, programs) are properly acknowledged at any point of their use. A list of the references employed is included.
Signed: Max Ferstenberg

Date: 24/03/2025

ACKNOWLEDGEMENTS

I would like to thank my supervisor, Oleksandr Letychevskyi, for providing timely and diligent guidance throughout the process of this project.

Table of Contents

List of Figures

List of Tables

Acronyms and Abbreviations

DNN: Deep Neural Network
IDS: Intrusion Detection System
NIDS: Network-based Intrusion Detection System
HIDS: Host-based Intrusion Detection System
LEO: Low Earth Orbit
LENS: Low Earth Network Segment
LISL: Laser Inter-Satellite Link
PoPs: Points of Presence
MAC: Medium Access Controller
IP: Internet Protocol
IPoS: Internet Protocol over Satellite
CCSDS: Consultative Committee for Space Data Systems
FSO: Free Space Optics
TCP: Transmission Control Protocol
UDP: User Datagram Protocol
RAW: Raw Socket
IRTT: Initial Round Trip Time
CSV: Comma-Separated Values
GRU: Gated Recurrent Unit
RNN: Recurrent Neural Network
GNU: GNU's Not Unix
MiTM: Man-in-the-Middle
DDoS: Distributed Denial of Service
DoS: Denial of Service
ROC: Receiver Operating Characteristic
AUC: Area Under the Curve
GDPR: General Data Protection Regulation
OSI: Open Systems Interconnection
GDB: GNU Project Debugger
PCAP: Packet Capture
VM: Virtual Machine
SSL: Secure Sockets Layer
TLS: Transport Layer Security
XSS: Cross-site Scripting
SQL: Structured Query Language
SSH: Secure Shell
FTP: File Transfer Protocol
IoT: Internet of Things

SMOTE: Synthetic Minority Over-sampling Technique
PEP-8: Python Enhancement Proposal 8
GCC: GNU Compiler Collection
CIC-IDS2017: Canadian Institute of Cyber Security Intrusion Detection System 2017 dataset
UNSW-NB15: University of New South Wales - Network Benchmark 2015 dataset

1 INTRODUCTION

1.1 Motivation

As reliance on and development of LEO satellites for global communication and internet connectivity increases, so do the unique security challenges in these systems. LEO satellite systems are quickly becoming more attractive targets for cyber-attacks due to the sensitive personal information they can store and their relative ease of access for hackers and external threats. This project aims to mitigate some of these threats and maintain secure global communications by implementing a new deep packet analysis technique for enhanced attack detection as the importance of global connectivity increases each year.

Traditional IDS are insufficient in terrestrial networks, a problem only exacerbated by the challenges faced by LEO satellites. To take a step towards fixing this problem, we aim to design a DNN tailored to the constraints posed by a satellite environment and study the effects of training a DNN-based IDS with an augmented input vector enriched with features extracted using GDB: The GNU Project Debugger. These features describe the internal state of programs and hardware, and this project aims to evaluate the addition of GDB-augmented features in terms of improving upon the shortcomings of contemporary IDS.

1.2 Aim and Objectives

1.2.1 Aim

The goal of this project is to design and implement two DNN models for signature-based intrusion detection and classification. These DNN architectures will be based on environmental constraints posed by Starlink LEO Satellites. We will focus on the effects of augmenting our input vector with low-level features describing the internal state of hardware and its system processes. Then, we aim to assess our augmentation in terms of the classic shortcomings of modern IDS: The false-positive fallacy and novel attack detection.

1.2.2 Objectives

The project aim will be achieved with the following broad objectives:

1. Identify the limitations of modern IDS and the constraints of our working environment.
2. Design and develop an initial DNN capable of processing traffic data and detecting attack signatures within these constraints, providing an accurate baseline representative of a standard IDS.
3. GDB-based feature extraction, augmenting our traffic data with a new method of deep packet analysis as packets are processed through a restricted launch of our working environment.
4. Experimentation of both the DNN model's architectures and parameters through

iterative improvement.
5. Evaluation of our models based on how well our augmentations address the false-positive fallacy and novel attack detection issue.

1.3 Contributions

The contributions of this project are as follows:

1. A DNN architecture designed for a satellite environment.
2. Assessment of a GDB-augmented input vector with features representing internal system information to an IDS model.
3. Performance evaluation of IDS after augmentation, addressing the shortcomings of IDS.

1.4 Organisation

In Chapter 2, we will review the background of our operating environment and its limitations. We will also discuss the limitations of current IDS and how our proposed augmentation methods will help circumvent these limitations.

Then, in Chapter 3, we will review our methodology to achieve this project's aims. This section will cover creating our testing environment, designing and implementing our DNNs, and data augmentation. Following our methodology, in Chapter 4, we will discuss how we will propose metrics and an evaluation schema for assessing the effectiveness of our two DNN models and the impacts of restricted launch feature extraction.

Chapter 5 will cover our implementation of the proposed methodology, the challenges faced during the process, and how these challenges were overcome. Lastly, in Chapter 6, we will comprehensively evaluate our implementation using our proposed metrics

2 BACKGROUND

2.1 Introduction

In this section, we will cover the underlying hardware and systems that support the Starlink environment encapsulated in the LENS dataset. We will then discuss the specific threats that satellites face in terms of cyber-attacks and how we can classify them. Finally, we will review and discuss current IDS systems and their advantages and shortcomings to conclude how the proposed system in this paper must operate and by what metrics its success should be measured.

2.2 Starlink Topology

The environment the data within the LENS dataset pertains to is Starlink, SpaceX's LEO network. In this section, we will cover the topology of this network and adapt it into a high-level interaction diagram, which we will then use to illustrate and model a functional environment for testing and training.

Starlink consists of four key points: satellites in orbit, User terminals/Dishes, Ground Stations, and Points of Presence (PoPs). Starlink user terminals, or "dishes," are phased-array antennas. They are constrained by an elevation angle requirement of 25 degrees, meaning they can only connect with satellites above this angle on the horizon. Each terminal connects to only one satellite at a time to ensure traffic forwarding integrity. The specifics of how terminals map to satellites are proprietary to Starlink, though it has been theorised how to model this mapping, covered later in this section (Tanveer et al., 2023). Starlink satellites manage connections with multiple user terminals simultaneously. Radioframes (bandwidth units) are allocated to a terminal managed by an onboard medium access controller (MAC) scheduler. Ground stations and PoPs are also equipped with phased-array antennas, which relay data to the network's PoPs. PoPs are servers with connections to the internet backbone, routing data to its final destination on the internet (Tanveer et al., 2023).

In terms of physical topology, Starlink uses laser inter-satellite links, which (LISLs) are divided into classifications (Chaudhry and Yanikomeroglu, 2021), as a summary:

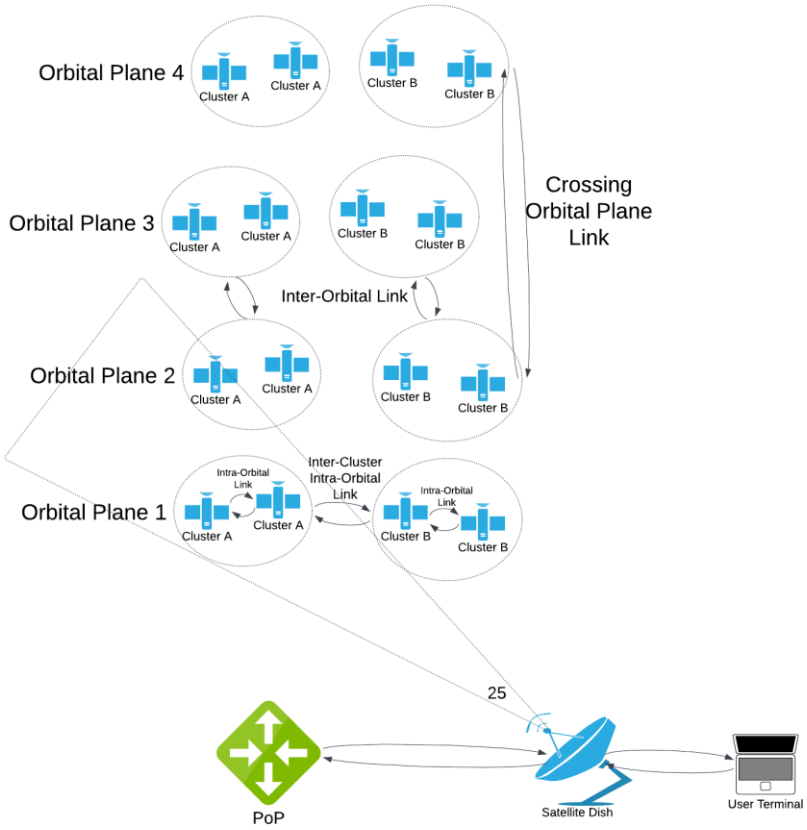
1. Intra-orbital plane LISLs are links between satellites in the same orbital plane (i.e. satellites with the same velocity).
2. Inter-orbital plane LISLs are links between satellites in different orbital planes and can be categorised into the following: Adjacent orbital plane LISLs, established between satellites in adjacent planes; nearby Orbital Plane LISLs, existing between satellites in orbital planes that are close but not adjacent and crossing orbital plane LISLs, linking satellites to each other when crossing orbital planes.

Satellites in adjacent or nearby planes have slight directional differences, resulting in varying relative velocities. These two classifications of LISLs mean that the topology forms two meshes. Regarding the environmental model we aim to create, LISLs introduce the conditions for latency and packet loss we face; crossing LISL meshes creates a potential latency spike or period of signal degradation (IMPROVING STARLINK'S LATENCY, n.d.).

Starlink uses a global scheduling system that assigns satellites to user terminals. Assignments are reallocated every 15 seconds, as observed via drastic latency changes across different vantage points (Tanveer et al., 2023). For the purposes of our environment simulation, this can be modelled by probabilistically assigning configurations of IP addresses and routing so that we can effectively apply the impact the scheduler would have on our environment.

The scheduler clusters satellites based on the following parameters: azimuth angle, elevation angle, satellite age, and sunlit status. For clustering, the model proposed by Tanveer et al. calculates how far each satellite's features are from the constellation's group mean and uses these to identify clusters. For example, if a satellite is further than one standard deviation on azimuth or elevation from the mean, it is grouped as such. In our model, we will pre-define address and port configurations to alternate with fixed probability to emulate the effect of the scheduler. Then, we will simulate "handoffs" between satellites with different velocities, which will adequately simulate the LISLs.

From this information, we can adapt our DNN architecture to accommodate these challenges, which allows us to train the DNN on data accurate to a Starlink environment.



(Figure 1 - High-level hardware environment interaction diagram)

Lastly, we will cover protocols. LEO satellites tend to use protocols defined by the Consultative Committee for Space Data Systems (CCSDS), which standardises data transmission protocols for space missions.

- CCSDS 133.0-B-1 is a protocol for transferring application data between satellites and ground stations (CCSDS Historical Document).
- Free Space Optical (FSO) communication, which uses laser beams to transmit data between satellites with high data rates and low latency. (Rani 2017)
- IPoS (IP over Satellite) is a protocol that encapsulates IP packets within satellite communication protocols. It integrates satellite networks with terrestrial IP networks, operates on top of standard IP protocols (IPv4, IPv6), and uses standard transport protocols used by IP (TCP, UDP) (Sanctis et al., 2016).

2.3 Intrusions in the Satellite Environment

Now that we have modelled the environment in broad terms in Chapter 2.2, we can understand the points of attack and attack vectors to which the environment is vulnerable, which helps us decide which attacks our DNN should be trained on. Satellites' physical distance and inaccessibility necessitate reliance on long-distance communication links and broadcast-based communication. This makes LEO satellite networks more vulnerable to specific attack vectors that compromise these communication channels. In this section, we will cover the specific intrusions and attack vectors that are prevalent in LEO networks.

LEO satellites are particularly susceptible to signal spoofing, replay attacks, and communication disruptions during frequent handovers. Inexpensive radio hardware is now more easily accessible to the public, making it easier for malicious actors to perform these attacks, intercepting and manipulating communications or disrupting services using these vectors (Smailes, 2023).

LEO satellites' inherent characteristics, like their low altitude and rapid orbital periods also contribute to vulnerability and open up additional attack vectors. Low altitude makes them more vulnerable to ground-based attacks, such as jamming and spoofing (Khan et al., 2022; Al-Hraishawi et al., 2021). As discussed in Chapter 2.2, quick and efficient routing protocols are also necessitated, which, if inadequately secured, can be exploited by attackers to redirect or intercept data (Chen et al., 2022). Additionally, the physical density of satellites creates vulnerability. As more satellites enter orbit, the crowded environment can lead to more challenges in maintaining secure communication links, as satellites frequently “hand over” connections to maintain coverage (Liu et al., 2021; Leyva-Mayorga et al., 2020). During the handover, data can be more susceptible to interception or manipulation (Park & Kim, 2021).

2.4 Semantics of Attacks

Now that we have discussed attack vectors within our environment in Chapter 2.3, we can understand the semantics of specific attacks included in our datasets and briefly discuss their potential consequences/effects in relation to our operational environment.

Heartbleed: A vulnerability in the OpenSSL library that allows attackers to read the memory of systems protected by SSL/TLS. Heartbleed attacks explot cryptographic weaknesses to feign access to sensitive data. Any satellites using SSL/TLS as part of IPoS could be compromised, (Synopsys, heartbleed.com).

Brute Force: This attack uses automated tools to guess passwords or credentials by systematically trying combinations. This involves the exploitation of weak passwords and poor security practices, potentially leading to access to interfaces managing satellite operations (Fortinet).

Web Attack - SQL Injection: Inserting SQL queries into input fields with malicious intent

to manipulate or compromise databases. This exploits database vulnerabilities, and within a satellite network, if ground control systems use databases, an SQL injection could lead to data corruption or unauthorised access. (Abdullah et al., 2022)
Web Attack - XSS: Cross-site scripting (XSS) attacks inject malicious scripts into web pages viewed by other users. This exploits web application vulnerabilities to execute scripts in the context of a user session. Such attacks would compromise web applications used for monitoring or controlling satellite operations.
Bot: This refers to automated scripts that perform tasks online. Bots can be exploited for malicious purposes like spamming or DDoS. They can create a large volume of automated requests, overwhelming systems.
Recon/Scanning: This indicates potential vulnerabilities and attack surfaces. Scanning ground stations could reveal exploitable services. (Muhmmad Ijaz Ul Haq, 2021)
DoS/DDoS: These involve systems (often botnets) where numerous compromised devices work together to flood the target. Their distributed nature makes them far more difficult to block (Fortinet).
Exploits: An exploit is a piece of software, chunk of data or sequence of commands that takes advantage of a vulnerability in software or hardware. Both satellites and ground stations rely on complex systems, with potential vulnerabilities in ground control systems or communication protocols (Team, 2020).
Fuzzers: Fuzzing is an automated software testing mechanism that involves providing invalid, unexpected, or random data as inputs to a program to discover coding errors or security vulnerabilities. Within a satellite environment, it could help identify vulnerabilities in telemetry and command systems, data processing software, or communications interfaces (Miller, Fredriksen and So, 1990).
Analysis: Analysis involves systematic examination of data, systems, and events to identify patterns and anomalies and reveal weaknesses in implementations. Satellite systems could be vulnerable to the analysis of protocols and side channels such as power consumption, electromagnetic emissions, or timing variations (Kocher, Jaffe and Jun, 1999).
It should be noted that this list is not exhaustive but covers all the relevant families of attacks and their semantics.

2.5 Intrusion Detection Systems

In this section, we provide an overview of IDS, existing systems, their limitations, and the reasons why they are insufficient. Then, we will discuss the environmental challenges IDS face in the satellite environment. Lastly, we will go over the data augmentation techniques we will use and their basis in IDS.

2.5.1 Overview of IDS

Intrusion Detection Systems (IDS) are components of network security that monitor traffic

and analyse it for signs of malicious activity or policy violations. The primary function of an IDS is to detect unauthorised access and potential threats to the system's confidentiality, integrity and availability. As Heady et al. 1990 defined, an intrusion encompasses any actions that compromise these security principles, making IDS essential in safeguarding digital environments. (Pachghare et al., 2012).

IDS are primarily categorised into Network IDS (NIDS) and Host-based IDS (HIDS). NIDS monitor traffic across several hosts, performing packet data analysis to identify suspicious patterns or signatures in traffic metrics. HIDS focuses on individual host machines, examining system calls and logs to detect anomalies in behaviour indicative of malicious actors (Elsadai et al., 2019; Tejakshi et al., 2018). Within these primary categories are several detection methodologies, including signature-based and anomaly-based detection. The former performs analysis of traffic against known attack patterns, whereas the latter identifies deviations from established norms (Liu & Liu, 2019; Zhong et al., 2020). The IDS we propose is a hybrid model that analyses traffic metrics in conjunction with host-based data extracted via GDB, using signature-based deep packet analysis for detection.

2.5.2 Limitations of Traditional IDS

IDS are not sufficient for attack detection or classification on modern networks. For example, many rely on signature-based detection methods, which, while helpful in identifying known threats, struggle more against novel attacks. This limitation is especially noticeable in the modern day due to the sheer complexity and quantity of interconnected devices, where the diversity of potential attacks is simply too broad for a signature-based IDS to learn every one of them (Khraisat et al., 2019).

On top of this, current IDS tend to generate high amounts of false positive alerts, which in and of itself isn’t a huge problem so long as the false negative rate remains low. However, the problem arises that external forces such as security personnel or even onboard computational resources have a much more significant burden, as they must spend extra time locating which alerts are genuine threats (Ryu & Rhee, 2008; Cuppens et al., n.d. Axelsson, 2000). The base-rate fallacy, discussed by Axelsson, also complicates the issue further, suggesting that inherent IDS characteristics could lead to alert misinterpretations, diminishing the significance or making them seem like more significant threats. (Axelsson, 2000).

Although these problems apply to all IDS, a model based on a terrestrial network would face some significant inadequacies within our environmental constraints. We will now go over some of these constraints.

Firstly, as discussed in Chapter 2.3, the rapid movement of satellites results in challenges with communication (variable link quality, high handover rates), which complicate establishing stable communication links (Liu et al., 2021; Leyva-Mayorga et al., 2020). Traditional IDS are typically reliant on stable network conditions when analysing traffic in order to detect anomalies; however, LEO satellite communications by nature can lead to incomplete data collection and, therefore, analysis, which could undermine the effectiveness of traditional IDS (Oligeri et al., 2023; Zhang et al., 2022).

Additionally, the space environment poses additional risks not addressed by conventional IDS. LEO satellites are susceptible to interference from outside forces, such as electromagnetic interference from solar activity or space debris, which can disrupt communication and data integrity (Ahmad et al., 2018; Vasylyev et al., 2019).

Satellite networks contend with latency and bandwidth constraints because of the environment in space. Existing IDS often depend on extensive data transmission for threat detection; in LEO networks, communication windows are limited, and bandwidth is at a premium (Wrona, 2023; Raja et al., 2020). Consequently, our IDS must operate within these constraints by limiting the quantity of data that it must analyse.

Finally, LEO satellites tend to have limited onboard computational resources, affecting the sophistication available to a potential IDS. Many traditional IDS rely on extensive computational power for real-time data analysis and machine learning. (Li et al., 2022; Guoyan et al., 2023). This particularly exacerbates the false positive issue; more of the already limited resources have to be allocated to misclassified benign traffic. Due to this limitation, we must develop lightweight, efficient IDS, minimising unnecessary computational power.

2.5.3 IDS for Satellite Networks

In this section, we will review how existing research proposes adapting traditional IDS for satellite environments, allowing IDS to function within the constraints and limitations faced by traditional IDS. As we have modelled the satellite environment in Chapter 2.2, we can use that as a baseline to understand how the proposed IDS will operate computationally.

Collaborative interference avoidance is the technique of sharing important information between satellites in orbit to improve the situational awareness of satellites without using unnecessary resources by transmitting redundant information. This could involve sharing the learned weights of a DNN to improve the detection rates of IDS on other satellites. (He, 2024)

The integration of multiple data sources enables an IDS to gain a more comprehensive understanding of the operational environment.

Edge computing is becoming increasingly prevalent in LEO satellite systems. Processing data closer to the source reduces latency and bandwidth usage, which is highly important in the resource-constrained environment of LEO satellites. An edge computing approach allows for real-time analysis of data streams for intrusion detection, which lowers response time to potential threats (Leyva-Mayorga et al., 2020).

2.6 Critical Analysis and Discussion

In this section, we will discuss the existing literature and background, its suitability for adaptation into methodology, and any compromises that must be made.

In terms of the literature surrounding Starlink, since Starlink is the most publicly accessible satellite environment, there is a substantial amount of satellite-related research on the subject. This made it easier during the research process to gather technological information.

However, due to the proprietary nature of many of Starlink's internal workings, the existing technology information does not include security systems and exact hardware specifications as we would ideally have for this project. As such, we will have to make the following concessions: The current security mechanisms of Starlink are not publicly available information, so we must work from the ground up. Additionally, the internal scheduler system is unknown, so the information we use here is theoretically derived. However, using publicly available information and the research posited by Tanveer et al. (2023), we can create a sufficient environment for testing and training. Since this project focuses on creating an IDS for the environment, we do not need to create a completely real-world accurate model. We only need to be able to create a model encapsulating the key characteristics of the environment. For example, we will not be able to map the global scheduler system entirely accurately. However, we can predefine clusters and use a probability-based mechanism to select them, allowing the IDS to work with a more realistic scenario.

The attack vectors discussed in Chapter 2.3 have proven to have real-world consequences, and we will examine some case studies to assess the importance and validity of these attack vectors in relation to the proposed DNN, which will be trained on them. Signal spoofing has been shown to be capable of disrupting a satellite's navigation system by using commercially available radio equipment. As discussed in a study by Salkeild, there is a necessity for security precautions against these kinds of attacks (Salkeild, 2023). Additionally, in 2022, some Starlink terminals operating in Ukraine fell victim to a jamming attack, demonstrating that this particular attack vector also has real-world applications (Foust & Berger, 2023). The

documented cases of attacks highlight the need for security systems that take into account environmental factors.

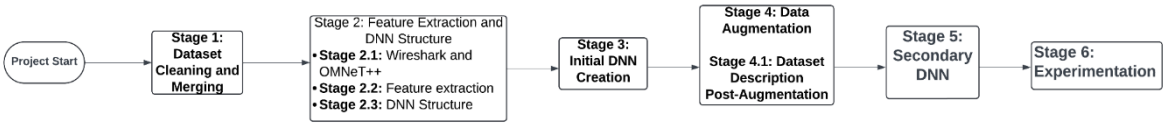
2.7 Summary

In summary, Chapter 2.2 discusses the topology and protocol information of our working environment, enabling us to create an accurate environment for augmentation. Chapter 2.3 discusses specific vulnerabilities to which this environment is exposed, and then we relate these vulnerabilities to the semantics of attacks contained within our datasets in Chapter 2.4. Chapter 2.5 discusses the main flaws of contemporary IDS. Primarily, modern IDS suffer from low reliability in detecting novel attacks and a high false positive rate. Finally, in chapter 2.5.3, we have explicitly discussed, within our satellite environment, the need for computationally efficient DNNs that minimise network resource use. Next, in our methodology, we will discuss our plan for accommodating these constraints and limitations.

3 METHODOLOGY

This section outlines the research and project methodology in stages. We will cover the technologies used, methods of exploration, and research performed, starting with a brief description of our datasets:

LENS LEO Network Traffic Dataset: The LENS dataset comprises satellite traffic metrics, primarily in RAW IRTT format, with limited latency data available in CSV format. This dataset provides our base case for satellite traffic data.
CIC-IDS2017, UNSW-NB15 and Kitsune Network Attack dataset: Chapter 2.4 contains a generalised list of attacks contained in all three datasets. These datasets contain signatures of attacks through network metric features, as well as PCAP files for all encapsulated traffic.
The below diagram shows the flow of our methodology. The stages will be completed sequentially.



(Figure 2: High-Level Methodology Plan)

3.1 Dataset Cleaning and Merging

Four datasets are the input for this stage. The first dataset, LENS, covers traffic metrics in two formats: RAW IRTT and CSV. This data represents the environment in which the IDS will operate, as modelled in Chapter 2.2. Then, we have three labelled attack datasets, which

contain network metrics that indicate the signatures of these attacks.

At this stage, we will merge the datasets to align protocol-specific traffic from the LENS dataset with attack vectors that are aligned with those protocols in the attack data. Then, we will clean the datasets so the DNNs will not learn incorrect patterns. Starting with the LENS dataset, the cleaning process primarily involves handling any missing values that may occur, especially in network datasets.

As discussed, our attack data comes from three different sources in order to cover a wide range of attack vectors while ensuring we have metric signatures for traffic most prevalent in the satellite environment. The cleaning process for the attack datasets is much simpler; it involves just handling missing values as before and then matching similar features across the attack datasets to merge into one.

The output of this stage is a clean and organised dataset attack data that are ready for Wireshark and, later, feature extraction.

3.2 Feature Extraction

3.2.1 Wireshark and OMNeT++

The cleaned and organised dataset from stage 3.1 is the input for this stage. Firstly, we will use OMNeT++ to create a testing environment based on our understanding of the satellite environment and Figure 1 in Chapter 2.2. OMNeT++ is an open-source network modelling tool that we can use to map how the packets will move between points in our network; the purpose of this is to train the DNN on data from an environment that is as accurate as possible.

The first step of this process is setting up our network topology. We can use nodes to represent satellite links, ground stations, and end-user devices. Next, we can use the INET Framework, a collection of pre-built models for protocols and networks, including satellite network protocols. This will also allow us to model LISLs between satellites. We will also need to define some internal mechanisms to ensure that data flows accurately and that the features we extract are as accurate as possible. The network scheduler discussed in Chapter 2.2 can be modelled by predefining clusters as configurations of IP addresses and assigning probabilities with a controller script to switch between these configurations. This approach will allow us to set up Wireshark and, later, GDB at realistic points that will be representative of how packets behave whilst maintaining minimal complexity.

OMNeT++ also directly integrates with Wireshark, which is a tool that allows us to inspect the packet data from our datasets as it travels around our mapped topology. Wireshark will

order the packets based on time, which is incredibly useful as it lets us see the sequence of events associated with both benign and malicious traffic. Additionally, Wireshark allows us to decode the usually unreadable information contained in packet layers, which we will discuss further in Chapter 3.2.2. Thus, this stage's output is our testing environment, and the packet data in a readable format.

3.2.2 Feature Extraction

The inputs for this stage are the packet information we have gathered from Wireshark. In this stage, we will perform feature extraction on the combined dataset. The goal of feature extraction is to extend the existing features in the dataset with sequential packet information for each request. This would include layer-level information within packets. From the data link layer, we can extract information such as source and destination MAC addresses, payload type and frame length. The network layer contains IP header information. The transport layer contains TCP flags and port numbers, while the application layer comprises HTTP headers and SSL/TLS information (Raza, M., 2018).

It is essential to note that, due to our datasets being sourced from different sources, some of the datasets already contain this information. However, through the feature extraction process, we aim to ensure that all our datasets are both aligned with the same information and supplemented with additional information. The features present in the LENS dataset prior to extraction will be used to provide baseline data for any features we extract. Thus, the output for this stage is our complete dataset with additional features.

3.2.3 DNN Structure

Due to limitations in our environment, discussed in Chapter 2.5.2, we need to discuss some architectural considerations to accommodate the environmental constraints. Firstly, due to the limited onboard computational power of LEO satellites, our DNN will have to be relatively shallow. While it would be impossible to determine exactly how many neurons each layer will contain at this stage, we aim to use as few neurons as possible while maintaining high accuracy. This will be discussed further in Chapter 3.6.

Due to the sequential and ordered nature of our data, an appropriate architecture choice for our DNN iss a Recurrent Neural Network (RNN). Traditional feedforward networks treat each input independently, but RNNs take into account the temporal sequence of data (IBM, 2023). While RNNs can handle sequential data, they inherently only handle short-term memory efficiently, so they struggle with learning from earlier time steps. To mitigate this, we can use an alternative architecture called Gated Recurrent Units (GRUs). GRUs use internal units called "gates" to determine which temporal information should be held in long-term memory in a lightweight, computationally efficient manner. This architecture is

important since it will allow the model to “remember” previous packets in a sequence and learn correlations that occur across multiple time steps (Weerakody et al., 2022; Bravo, 2021).

As discussed in Chapter 2.5.2, satellite environments face frequent link changes and handovers, as well as latency fluctuations due to satellite movement. To adapt to this, we can introduce attention mechanisms. Attention mechanisms essentially assign weights to input elements in the sequence as the model trains so that it can prioritise which features are most important. Additionally, to adapt to this unreliable environment, we can use dropout layers. Dropout layers randomly “drop out” (set to zero) a number of neurons in the network during each training iteration. This means that each time the model sees data, it trains on a different subset of neurons, which forces it to be less reliant on specific neurons. This is important because it means that even with satellite data being highly variable and sometimes noisy, the model will be more adaptable. This structure is based on existing DNN IDS structures (Tang et al., 2016) and architectural considerations for our specific environment.

As discussed above, the input layer for our DNN will likely contain 30+ neurons due to the large number of features we are dealing with. For multi-class classification, we will use SoftMax as our activation function. SoftMax is also appropriate since it outputs probabilistic classifications (Goodfellow, Bengio, and Courville, 2016). This approach works for identifying novel attacks, as it does not classify attacks absolutely; instead, it outputs what the attack is most likely to be or most similar to (Shah, D., 2023). Therefore, the output for this stage is our DNN structure, which we can now implement.

3.3 Initial DNN Creation

This stage takes as input the complete dataset after feature extraction and the DNN design from Chapter 3.2.3. In this stage, we will implement the first DNN model in Python.

In addition to our architectural considerations, several implementation considerations also need to be taken into account. Firstly, to keep up with limited computational resources, we can reduce the amount of memory that the model will use by applying quantisation. Typically, DNN models use a 32-bit floating-point to store model parameters (weights, biases, and activations), but quantisation converts them to 8-bit integers after training. This means that, with minimal impact on accuracy, the memory usage footprint of the IDS is reduced.

In terms of technologies, TensorFlow Lite is a lightweight training framework designed for edge computing. As discussed in Chapter 2.5.2, edge computing is important for reducing latency and bandwidth usage. The loss function we will initially implement will be

categorical cross-entropy. Categorical cross-entropy is widely used for multi-class classification and encourages the model to output a probability distribution, similar to our reasoning for utilizing SoftMax.

Finally, because the satellite network consists of multiple devices linked to each other, when implementing, we can design the model for federated learning by ensuring learned weights are shareable. Federated learning is an approach to machine learning in which multiple devices collaboratively train a model without sharing raw data. Instead, they perform computations locally and share model updates, such as weights and gradients, with the central server (Shastri, Y 2023). In our case, the central server will be the ground station. Federated learning means that the IDS will be a lot more bandwidth-efficient, and it will make sure that computational resources are not wasted on sending large amounts of data between satellites. This also accounts for the idea of collaborative interference discussed in Chapter 2.5.3 (Shastri, Y 2023).

The output of this stage is our initial DNN, implemented with consideration for our environmental requirements.

3.4 Data Augmentation

In this stage, the input is the full dataset post-feature extraction. This stage describes a method of data augmentation to extract hidden layer information from packets and otherwise inaccessible hardware information.

To extract hidden layer information, we will use GDB: The GNU project debugger, to run our existing data in a restricted launch mode. A restricted launch allows us to pause execution at defined breakpoints (Free Software Foundation, 2024). In a later stage, we will utilize these additional features to train a secondary DNN on more complex low-level features, enabling a new type of deep packet analysis for neural classification. Thus, it is important to note that we do not implicitly need to understand the relations these features represent because the secondary DNN will do that for us.

As for the actual process of doing this, firstly, we will need to attach GDB to a running process; in our case, this would be our OMNeT++ environment. This will launch the OMNeT++ environment in restricted mode, allowing us to place breakpoints on processes within satellites, user terminals, etc. When the program runs and hits these breakpoints, it pauses so that we can inspect their internal state. At this point, we are able to extract the hidden layer information (Free Software Foundation, 2024). To streamline this process, we can set up a GDB script that will automatically allow the program to run and output our extracted information.

Regarding suitability for GDB usage, while it is not typical for GDB to attach to a program like OMNeT++, it does work for our purposes. This is because of how OMNeT++ itself operates, in that it is based on C++. Memory management in C++ is done manually, so we can inspect memory regions directly; as packets are run through OMNeT++, they exist in memory as objects that represent attributes like headers or payloads, so with GDB, we can directly examine these objects and place breakpoints within them (Free Software Foundation, 2024).

Thus, the final output for this stage is a dataset with low-level features for deep packet analysis that we can use to train our secondary DNN.

3.4.1 Post-Augmentation Dataset Description

Now, let's briefly discuss what our dataset looks like after augmentation. Firstly, the actual features we are extracting are low-level system information such as memory registers, processor states or call stacks. (Free Software Foundation, 2024). Our dataset will be extended with these features, captured at the point of packet acceptance into a satellite within our network, which, in sequence, provides us with a deep analysis of sequential packet information from both benign and malicious traffic. The false-positive fallacy we have discussed arises from the extreme class imbalance, where almost all network traffic is benign (Schneier, 2024). We aim to address this issue, as well as that of novel attack detection, by enriching our DNN input vector with deep packet analysis.

3.5 Secondary DNN

At this stage, we have our augmented dataset as an input, and we will train a second DNN using the new dataset as the input. Architecturally, the primary difference between the first DNN and this one will be the number of neurons in the input layer, which accommodates the additional features, and potentially the number of neurons in the hidden layers, which scales with this change.

As discussed previously, augmentation extends the dataset with hidden layer features. In theory, this will allow our secondary DNN to recognise more complex, lower-level patterns that are representative of attacks. The goal of this is to help address the primary challenges of modern IDS: the inability to reliably identify novel attacks and high false positive rate. We will discuss how to appropriately evaluate the effectiveness of our approach in Chapters 3.6 and 4.

This secondary DNN will be used to evaluate the effectiveness of our data augmentation in compensating for the inadequacies of modern IDS. The output for this stage will be our

second DNN model trained on augmented data.

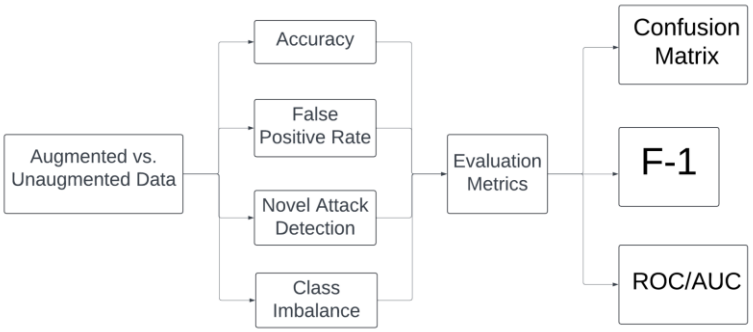
3.6 Experimentation

At this stage, our inputs are the two DNNs we now have: One trained on unaugmented data and one trained on augmented data. We are now ready to start experimenting with parameters and architectures on our DNNs. This stage goes hand in hand with Chapter 4, as we now need to evaluate the effectiveness of any changes we make. As discussed in Chapters 3.2.3 and 3.3, we need our DNNs to operate with the highest accuracy possible while maintaining as few layers and neurons as possible. In addition to architecture, we can experiment with different predefined loss functions. This process will involve running both of our models repeatedly, logging our results and evaluating our results based on the metrics described in Chapter 4.

Of course, the main objective of experimentation and evaluation here is to find the effects of data augmentation on the IDS. To this end, we can use the same evaluation metrics on our two models after we have found their optimal architectures.

Thus, at the end of our methodology, we have two DNN models to compare and evaluate according to the evaluation metrics that we will now explore.

4 EVALUATION STRATEGY



(Figure 3: Evaluation Schema)

As previously discussed, we are evaluating our two DNNs against each other to assess the effects of augmentation. Firstly, let's lay out our evaluation goals:

1. Determine how accurately each IDS can classify traffic as benign or malicious
2. Evaluate model accuracy on novel attacks
3. Assess how augmentation affects class imbalance
4. Evaluate how augmentation affects false positive/negative rates

We measure our goals using confusion matrices, F-1 scores, and AUC/ROC curves. To the end of evaluating IDS accuracy, we can lay out the following confusion matrix:

- The X-axis will be the true class, e.g. “Fuzzers, DDoS, Benign”, the class containing the labelled type of traffic
- The Y-axis will be the class that the model predicted (predicted class)

From this confusion matrix, we can classify a false positive as either benign traffic classified as an attack. Conversely, a false negative is when attacks are classified as benign. True positives would be occurrences where the presence of an attack and classification of the attack are correctly identified. In our case, the false positive rate is the metric we care about the most; the aim of this study is to improve upon the shortcomings of modern IDS, which, in a large part, is the false positive rate.

The F-1 score is the mean of precision and recall. Recall measures the proportion of true positives among all actual positives, while precision measures the proportion of true positives among all predicted positives. As discussed, a high false positive rate is one of the main failings of contemporary IDS, so through F-1, we can evaluate how well our IDS minimise false positive rates (precision) while maximising the detection of actual attacks (recall).

The ROC (Receiver Operating Characteristic) is a graph plotting recall against a false positive rate. The AUC (Area Under Curve) provides a single scalar value summarising the model’s ability to distinguish between classes; as a reference, an AUC of 0.5 means the model is 50% accurate, and an AUC of 1.0 means perfect classification. By comparing the F-1 and AUC scores of our two DNNs, we can evaluate their accuracy and assess how augmentation affects false positive rates.

As for evaluation against novel attacks, we take a simpler approach, since we only need to evaluate if the model identified the attack as any malicious class or not, so we have a simple subset of novel attack cases, taken from the most underrepresented classes in our dataset, calculating a simple percentage to identify effectiveness in novel attack detection; the DNN model would have no way of classifying a novel attack according to any meaningful class, since they will be unknown classes.

Lastly, to evaluate how augmentation addressed class imbalance, we can hone in on minority classes in the dataset and directly compare true positive rates between the models in predicting these classes, though we can also note that higher recall values in benign traffic also indicate improvements in false-positive rates.

5 IMPLEMENTATION

5.1 Dataset Construction

Before we discuss how we are constructing our dataset, let us look at its purpose:

- Provide a proportionally representative dataset of network traffic consisting of two parts:
  1. A dataset of network requests, labelled with lower-order metrics on a per-packet basis.
  2. A high-level dataset of network requests, labelled with higher-order metrics for the entire request.
- These two datasets are linked; individual packets in our first dataset are mapped to the request they belong to via a common request ID.
- Since our attack data was not measured from a satellite network, we need to provide all data in a manner such that all metrics are normalised with respect to characteristics of an LEO satellite network, i.e. characteristics such as loss or delay are all proportionally representative of values one would expect to see on such a network.
- Provide an identical version of the dataset where packet-by-packet breakdowns contain hidden features describing the satellite's physical internal state extracted via GDB at each point directly after the satellite processes a packet.

At this stage, our goal is to provide a complete, high-level dataset of network requests. The LENS LEO dataset is the cornerstone of our dataset construction process; it forms a baseline representation of the characteristics of satellite network traffic. This dataset required little cleaning, but the IRTT data had to be processed from JSON to CSV format. Algorithmically, the written Python script that performs this processing works as follows:

1. We flatten the JSON file's nested dictionaries into a single-level dictionary by concatenating keys with a separator indicated by the dictionary structure.
2. We read the JSON file's "header," which contains the overall metrics for the entire request, using IJSON and flatten it in a similar manner.
3. Take each round-trip entry (a dictionary) and construct a new dictionary with a fixed set of keys for values contained within it.
4. Write each round-trip entry to a CSV row, using the values within each dictionary as the field names for each column.

The object fields are too numerous to explain each and every one, but to outline how we are using this to support our larger methodology, we will go over some of the most relevant fields:

- **lost:** A count showing whether a packet was lost during the round trip. This metric can be used directly in our testing environment to simulate accurate parameters for further augmentation by setting percentage loss on links and calculating an overall

percentage loss rate.

- **delay_send/delay_receive:** The recorded delay at the time of packet send/capture is another metric we can use directly in our testing environment by setting delays on ports. We can also use this to ensure that any delays in our data not recorded from a satellite network can be scaled to have delays realistic to those that one might encounter in such a network.
- **ipdv_send/ipdv_receieve:** Inter-packet delay variation for packet send/capture, which we can use by setting jitter in our simulation. Similarly, we can adjust metrics of non-satellite data to match what you would expect to see in a satellite network.

Since each JSON file already contains metrics for every packet in a request, we don't need any additional work here. As stated previously, the purpose of this portion of the dataset is to provide a baseline against which to normalise all other traffic data. To this end, we need to calculate a constant for each feature by which we can normalise all other data. Our strategy for this is ratio normalisation. Since each window in which this dataset was measured is exactly one hour long (Zhao and Pan, 2024), we can use this fixed duration to calibrate our measurements. Our approach to this is as follows:

1. **Duration normalisation:** Since the duration of a satellite communication window is known to be 3600 seconds, we normalise the duration of operational flows by computing:

$$\boldsymbol{Normalised\ Duration} = \frac{\boldsymbol{Observed\ Duration}(\boldsymbol{s})}{\mathbf{3600}}$$

2. **Other Metrics:** Metrics such as bytes received, packet rates, delay, or jitter will be normalised according to a baseline Z-score from the LENS LEO dataset, and then operational measurements are expressed as a ratio relative to this baseline. For example:

$$\boldsymbol{Normalised\ Delay} = \frac{\boldsymbol{Delay}(\boldsymbol{s})}{\boldsymbol{Delay\ Baseline}}$$

Ratio normalisation guarantees that all network flows are evaluated relative to the fixed window, so our models will be trained on environmentally representative data.

The CIC-IDS2017 and UNSW-NB15 datasets form our attack data on which our DNNs will be trained. We follow a different process since they already come in CSV format. We map each field name in the attack datasets to each field name in the LENS LEO dataset, which results in some missing fields and values. In Chapter 7.2.2, we will detail how these values are filled and how we derive the packet-by-packet breakdown for each request. After we have derived these features and filled in our gaps, all values are normalised with the above strategy.

A challenge during this stage was that the Kitsune Network Attack dataset had to be dropped during the cleaning process. After downloading the KitNet framework, which is used to construct raw PCAP data into a labelled dataset, we can use it to extract the labels for each feature within the dataset; however, doing this revealed a significant issue that was not immediately obvious during our methodology plan; the features encapsulated in the Kitsune dataset are high-level abstractions of combined metrics. This means that without knowing the exact math used to construct this dataset, we cannot reliably group packets based on flows; thus, the dataset had to be dropped.

The output for this stage is a complete higher-level dataset of network requests, and we can move on to constructing our lower-level dataset of packet dissections.

5.2 Feature Extraction

In this stage, we will cover how we set up our testing environment, considerations for GDB usage, and how our packet-level datasets are fully constructed.

5.2.1 Wireshark, GNS3 and C

First, we will cover the model of our satellite network. The purpose of this network is to provide an accurate simulation of an LEO satellite environment. Our goal is twofold:

1. To construct a dataset of packet dissections that matches our high-level dataset of requests, mapping each packet to each request in flows.
2. Augment that dataset with features representing the satellite's internal state (extracted via GDB) during packet processing.

Our approach has evolved significantly from the original methodology plan. Initially, we intended to use OMNeT++; however, we transitioned to running docker containers on a GNS3 virtual machine. This shift was motivated for several reasons and provided us with many practical advantages: First and foremost, OMNeT++ has little in the way of continued support or documentation, which made any progress with it slow, and accurately interpreting our results was both difficult and time-consuming. A GNS3 VM, however, includes the integration of Python scripts, which in and of itself allows us to access a much broader range of libraries and tools, allowing us to use libraries such as Scapy and TShark, which makes data processing and interpretation far cleaner and faster. The specific usage of these libraries will be detailed in Chapter 7.2.2. Most importantly, however, while OMNeT++ does have native GDB integration and support, it does not allow for the same kind of GDB scripting that a VM does, which is crucial due to the volume of data we need to process through GDB; we want augmentation data for every individual packet.

Our initial docker network consisted of separate containers, representing our core nodes in

the topology described in Chapter 2.2: User Terminal, Satellite 1, Satellite 2, Ground Station, and Point of Presence. Subnets were allocated for each link between containers. A supervisor Python script was run in our VM that provided asynchronous calls to update the IP addresses of our two satellite containers every 15 seconds, mimicking the behaviour of the Starlink scheduler without the need for an unnecessary number of elements in our topology. The way this worked algorithmically was as follows:

1. Add new IP aliases on the containers from a pre-set dictionary of possible configurations (called dual homing)
2. Waiting for an overlap period where neither port is in use to let existing connections migrate
3. Remove the old IP addresses
4. Flush the neighbour cache and bounce the network interface to enforce the changes.

However, while the scheduler was able to make connection handoffs, it raised many complications when GDB was added into the mix. GDB attaches to a running process and allows one to place breakpoints within that process at specific function calls, but when a breakpoint is hit, the process to which GDB is attached halts. With the asynchronous calls running in the background, the process halting completely invalidated the scheduler's ability to make handoffs correctly; thus, the environment became unstable. Due to this, the scheduler was dropped in future iterations. The main reason for our understanding of the satellite environment and scheduler was to understand the specific challenges it faces, which have already been accounted for in our DNN architecture, and our satellite baseline data has already been taken from an environment where the scheduler is in effect. Thus, the scheduler's impact on our baseline data is already in place, and any impact on subsequent data is likely minimal.

Additionally, parameters for connections between containers were set from values read from our dataset according to each request as it was being replayed from the PCAP file using Tshark. This was done by parsing values into syntax for commands to alter container connection parameters. For example, to set a specific loss and delay:
docker exec SAT1 bash -c "tc qdisc add dev eth0 root netem delay 3.02ms 4.28ms loss 0.82%"

While this approach on its own did not yield any other significant difficulties in and of itself after dropping the scheduler, problems arose once we began to integrate potential server models. After undergoing extensive experimentation and attempted server setups using NetCat, Socat, and iPerf3, each of these tools yielded shortcomings that voided their suitability for our purposes:

- iPerf3: Although useful for testing connections, iPerf3 does not replicate the server

behaviour required for our analysis. It is designed primarily as a connection testing tool rather than a server model that processes and routes packets.

- NetCat and Socat: Both were considered alternatives but were ultimately dropped because their source code lacks the necessary debugging symbols and instrumentation points for the amount of control we require for GDB integration. Without the proper debugging symbols, GDB cannot accurately decode all the internal information from the operating system and, therefore, limits the features that we can add to our DNN input vector.

Finally, we landed on our solution: Constructing our own simulated server and environment in C. This server runs inside a single docker container so that we can still provide a restricted launch for GDB to operate in. Two key considerations drove the design of our C server, improving upon the shortcomings of our previous approaches:

1. Algorithmic Clarity: Since this is built from the ground up, there is no ambiguity in the source code as to where exactly operations are being performed.
2. GDB Integration: By constructing our own server, we have complete control over the source code; the necessary debugging symbols and instrumentation are available easily for GDB, and there is no ambiguity as to where GDB should be attached.

The C server processes raw PCAP files and applies known realistic delays, routing, and packet losses typically observed in satellite communications. The overall workflow is as follows:

1. It opens and reads PCAP files to extract raw network packets
2. Each packet is parsed to extract key network parameters such as source and destination IP addresses, ports and MAC addresses, as well as any payload information.
3. Depending on the packet's origin (uplink or downlink), the server applies realistic network delays (using *usleep*) and routing decisions.
4. The server simulates multiple hops between nodes (e.g. user terminal → satellite nodes → ground station) by recursively processing the packet through a simulated topology
5. A log of processed PCAP files is maintained to keep track of the sequence in which files were processed.

The environment consists of "nodes", represented implicitly as MAC address variables and function calls:

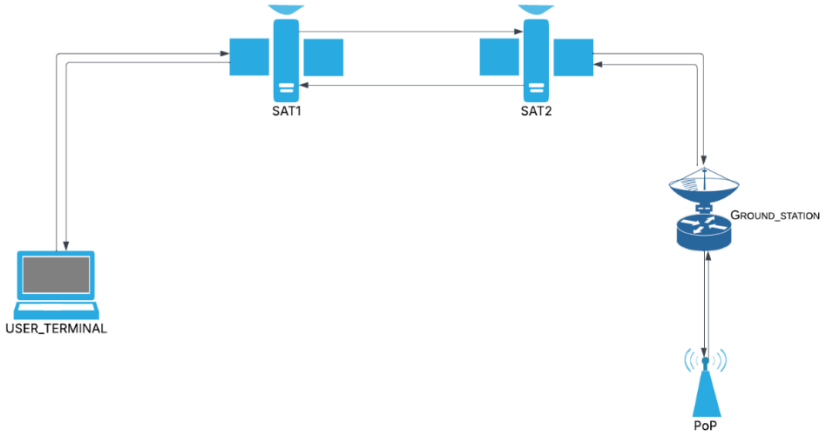
- User Terminal (UT): Originates network traffic
- Satellite Nodes (SAT1 and SAT2): Emulates LISLs and routing between satellites and ground stations
- Ground Station (GS) and Point of Presence (PoP): Represents gateway nodes

interfacing with external networks.

The server uses predefined MAC addresses for simplicity since the PCAP files already contain actual MAC addresses as part of their header information. Delays are simulated corresponding to each link:

- UT ↔ SAT: Uplink delay (25ms)
- SAT ↔ SAT: LISL delay (5ms)
- SAT ↔ GS: Downlink delay (25ms)

Below is a diagram illustrating the flow of data between our “nodes”:



(Figure 4: Simulated environment topology)

The C server relies on several standard libraries and tools, but the most specific ones for our purposes are:

- Networking Libraries:
  - <*netinet/tcp.h*> and <*netinet/udp.h*> for transport layer headers.
  - <*netinet/if_ether.h*> for Ethernet frame processing.
  - <*netinet/ip.h*> for extracting IP headers.
- <*pcap.h*> is a PCAP library for opening and reading PCAP files.

The server is compiled using a standard C compiler (GCC):
gcc -o server server.c -lpcap

In our environment, packets are read and stored into a defined packet structure object containing metadata for each packet as described in the PCAP file (IP and MAC addresses, ports, length, payload pointers, etc). Our packet parser function extracts MAC and IP

addresses from these structures and records the protocol field (TCP/UDP). The function also extracts relevant port numbers depending on whether the packet is TCP or UDP. A local instance of this function is called explicitly within the second satellite node (SAT2) because this is what we want to attach GDB to. In this position, we can see the exact impact of packets being parsed into memory addresses, local variables, arguments, etc. This will be detailed further in Chapter 4.

We also have a packet processor that directs the packet through the simulated network. If the packet reaches SAT2, it is passed into a specialised handler, as described above. Otherwise, the next hop is determined by our other *get_next_hop()* function, and the packet is forwarded by calling *simulate_link()*.

Before we describe how *get_next_hop()* and *simulate_link()* work, we have to determine the packet flow direction using a simple bool to determine the packet destination; if it is UT, then the packet is downlink, and if it is POP, the packet is uplink. Once this has been determined, the *get_next_hop()* function maps the current node's MAC address and the direction of flow (uplink/downlink) to the next hop's MAC address. Following this, *simulate_link()* applies an appropriate delay to the link, pausing execution briefly, and then a random value is compared to the packet loss probability. If the condition is met, the packet is dropped. Finally, the function recursively processes the packet by calling *process_packet()* with the next hop's MAC address.

Thus, the output for this stage is our functional satellite environment model.

5.2.2 Packet Feature Extraction

In this stage, our goal is twofold: First, we will fill in any missing values in our high-level request dataset left over from our initial merging and cleaning process. Second, we will create a complete low-level dataset of individual packets mapped according to their corresponding network request.

After making the shift to our C environment, ultimately, the process was heavily streamlined by focusing on post-capture processing. Python scripts were developed to directly parse PCAP files and map packets based on constructed unique tuple keys for each request. These keys consist of source and destination IP addresses and ports, protocol and timestamp, and a reverse key (the same key, but the source and destination are swapped) to indicate packets from the same request but flowing in the opposite direction. Algorithmically, the script reads an entry from our request dataset and constructs a unique key, and using *Scapy*, a Python Wireshark library, iterates through all packets in a PCAP file, comparing each packet's metadata to each request's constructed key. Matched packets are aggregated and grouped into

individual flows, then converted to CSV format and stored.

This mapping does two things for us:
Each packet in our dataset is now associated with a request based on its identifying tuple
Once the packets are grouped, we can perform mathematical derivations to calculate any features missing from requests as artefacts of the initial merging and cleaning process.

Derivations for missing features are calculated from our newly created CSV files for the packet breakdown of each request. To understand how we are deriving these features, let's look at what Wireshark actually tells us about packets:

At Layer 2 (Data Link), we identify source and destination MAC addresses, EtherType, and frame length. From this layer, we mostly care about frame length, which is used in our calculations for average request packet sizes. For example, the mean packet length (e.g., *smeansz* and *dmeansz*) is derived from frame lengths captured at this layer. (wiki.wireshark.org, n.d.)

At Layer 3 (Network), we can decode the IP header, including source and destination IP addresses, header length, and TTL (Time-To-Live). For our derivations, TTL values are used to derive features such as *STTL* and *DTTL* (representing TTL values on uplink/downlink). These values also indicate the number of hops a packet has taken. (wiki.wireshark.org, n.d.)

At Layer 4 (Transport), we can extract TCP/UDP ports, sequence numbers, window sizes, etc. Specifically, we use TCP flags to compute metrics such as *ACKDAT* (the delay between the initial packet and first acknowledgement) and *SYNACK* (the difference in timestamps between *SYN* and *ACK* packets). Together, these support our derivation of *TCP RTT* (TCP round-trip time). We can also analyse differences in timestamps from packets (based on sequence numbers and flags) to calculate inter-packet delay variations. (Wireshark.org, 2019)

At Layer 7 (Application), it can decode HTTP headers, DNS queries, SSL/TLS handshake details, etc. This is less relevant for us since our traffic is being run on a local simulated network. It doesn't make DNS requests outside of that network, nor does it need to make/request handshakes. (Wireshark, 2025)

Thus, moving out of this stage, our datasets pre-GDB augmentation are complete and ready for DNN training.

5.3 GDB Augmentation

Our approach to improving standard IDS capabilities' shortcomings revolves around adding

features that express the internal hardware and system state as our satellite deals with packets. To this end, we apply GDB: the GNU Project Debugger, in conjunction with our C environment. The primary function of GDB is to place “breakpoints” within a running process, essentially causing the process to halt when the subprocess we place a breakpoint at is reached. From here, the user can inspect the internal state of that process, as well as the platform on which the process is running (Sourceware.org, 2025). Unlike our earlier experiments with dockerised iPerf3, NetCat and Socat instances, our C environment and server provides us with a clear and purpose-built way to facilitate precise breakpoint placement and controlled execution.

As previously discussed, the decision to implement our environment in C is due to GDB’s native libraries and debugging symbols; GDB primarily interfaces with processes and the operating system using C (Sourceware.org, 2025).

In our case, we place a breakpoint directly on the local version of our *packet_parser()* process inside our SAT2 node in our network. This breakpoint is chosen so that it triggers at the point after the packet has been accepted, is about to be processed and before it is routed away to its next hop. From this point, we can step into the function, giving us access to the internal state of the program as packets are processed.

In order to do this, we have to run our environment in a restricted launch. The debugger launches the inferior (the program being debugged, the C server in our case) under a limited environment. In this mode, GDB intentionally restricts what the inferior process can do, essentially isolating it from outside interference (Sourceware.org, 2025). Externally, we run our SATNET docker container under *“--cap-drop=ALL and --cap-add=ADMIN”*, which drops all capabilities other than admin privileges to access the kernel. This is so that the container cannot perform operations that aren’t required for debugging. In addition, we set the kernel’s *ptrace_scope* to 1, which means that a process can only be traced if it is a direct child of the tracer, which, in our case, prevents arbitrary processes from being debugged (Docker Documentation, 2024). This also is the rationale behind doing this inside a docker container, inside the VM; doing it this way provides complete compartmentalisation from any outside interference. This setup allows us to safely attach GDB to our host process.

Every time the breakpoint is hit, the server halts, and GDB executes a series of commands. At this point, we have a few options for what we want to analyse. After experimenting with various features to extract from GDB, for our purposes, local variables, arguments and memory address states. Below is a sample of what this actually looks like.:

Local Variables:

```
(gdb) info locals
src_mac = "00:c1:b1:14:eb:31", dst_mac = "b8:ac:6f:36:08:f5"}
is_uplink = false
next_hop = 0x55555555604a "00:00:00:00:00:02"
pinfo = 0x7fffffffdec0
```

Arguments passed into the function:

```
(gdb) info args
{0x555555562ae0 "\270\254o6\b\365", 711, "172.217.7.132\000ca",
"192.168.10.8\000\00032", 6, 443, 50182, "00:c1:b1:14:eb:31",
"b8:ac:6f:36:08:f5"}
```

Local memory address state:

```
(gdb) x/32xb &pinfo->packet
0x555555562ae0: 0xb8  0xac  0x6f  0x36  0x08  0xf5  0x00  0xc1
0x555555562ae8: 0xb1  0x14  0xeb  0x31  0x08  0x00  0x45  0x00
...
```

The output from GDB is parsed and appended to our packet-by-packet CSV files. Each record now contains standard network metrics and has been augmented with these additional low-level features. In Chapter 5, we will detail how these enriched input vectors are used to train our RNN.

Given that our goal is to augment the input vector of our DNN with these features, let's discuss the potential impact of these features and how they might relate to specific attacks. Firstly, it should be prefaced that this is a partially speculative discussion for the purposes of explaining observed behaviours and values based on our environmental setup and foundational knowledge from the referenced papers and articles. Memory registers hold the most recently used instruction or location of a process; cyber attacks often cause large delays or irregular patterns in calls to processes (Liu et al., 2018)., for example, DDoS attacks essentially aim to deny access to a service by flooding that service with requests, and it is likely this would be noticeable in memory registers, as you would likely see repetitions of the same calls or addresses, or there would be large gaps in memory where a process has been held up elsewhere. Host-based IDS analyse auditing data from operating systems, and monitoring real-time system call traces is one established method of doing this (Liu et al., 2018); therefore, it is also likely that local variable values would also contain extreme values

and erroneous gaps in values, from information loaded in by or to system calls, which could be indicative of malicious traffic holding up processes.

Let's look at what we can view but is likely redundant or static due to our environmental setup. Understanding what we are not looking for will better illustrate GDB's purpose and specific usage for our methodology, as well as highlight some potential limitations of our environment and choice of breakpoint placement in the process. In our environment, processor register states largely don't change much from packet to packet. This is likely because within the satellite, the processor is operating at a somewhat higher level and will be calling mostly the same processes for accepting and sending packets repeatedly and does not show things like gaps in time or erroneous values. In a typical system, processor registers are used for low-level CPU operations, such as instruction execution and memory addressing (Silberschatz, Galvin and Gagne, 2018). However, in our code, the focus is on packet processing and replaying PCAP files, parsing packet headers and simulating network links. This design does not involve specific register manipulations from any instructions contained within the packet payload; satellites in our setup parse and route packets to their destination, rather than actually executing the instructions that could be contained within the packet; the purpose of an IDS is to stop malicious traffic before it reaches its destination. Therefore, at this level of observation, register changes may not be the most distinguishing feature. Global variables are mostly redundant information for a similar reason; the information here is too high-level and does not necessarily represent the state of the processes we are interested in. Global variables have a broad scope and are often used for configuration settings or program-wide state (Kernighan and Ritchie, 1988). In the context of network packet processing, these global values might include network interface status or simulation-wide parameters, which do not change dramatically with each packet and will produce large amounts of data, much of which will not be relevant. Our code reflects this, where global definitions like `UPLINK_DELAY_US` or `SAT1_MAC` are used for the overall simulation setup, not for per-packet state tracking. Other options for what we can view in GDB include the backtrace, which will be static at the breakpoint we are analysing. Given that we are focusing on the point immediately after packets are accepted, rather than a few syscalls down the line, the backtrace will consistently show the sequence of function calls leading to that specific point in the code. Finally, while assembly instructions provide a lower-level interpretation of the code, they offer more detail than necessary for our analysis; as discussed in Chapter 2.2, computational limitations of the satellite environment must be taken into account here, and the addition of assembly instructions into our input vector could potentially bloat the input vector by hundreds of features, many of which may end up being completely unnecessary, creating too much computational overhead. By instead examining the local variables within the functions relevant to our goals, we can glean a higher-level and more useful set of information. (Sourceware.org, 2025)

5.4 RNNs

5.4.1 RNN Architecture

As described in Chapter 3.3, our RNN is designed with two primary branches that process different aspects of our input. Firstly, we need a static branch for our high-level request metrics. In this branch, we can treat each feature as static since there will be only one input for the whole request. The input & dense layer is a vector representing more high-level metrics for each network request, with 32 neurons and using a ReLU activation function. This branch also uses a dropout layer with a rate of 0.3. Our second branch is for sequential features; the network accepts variable-length packet sequences, using a masking layer to handle padded zeros. Two GRU layers are employed in sequence (64 and 32 units, respectively) and capture temporal dependencies in sequenced data. Each GRU is paired with a dropout layer (rates of 0.4 and 0.3).

We also use an attention mechanism where a dot-product operation calculates similarity scores between GRU outputs, producing a self-attention score matrix. Attention score is essentially a computation of the "relevance" of each part of the packet sequence (Vaswani et al., 2017). The scores in this matrix are normalised using a SoftMax function, which should effectively "highlight" significant parts of the packet sequence (Bahdanau, Cho and Bengio, 2014). A second dot product combines attention weights with GRU output to generate a single, weighted sum: our context vector. This is followed by a global average pooling, which condenses the information into a fixed-length representation that can be fed into subsequent layers.

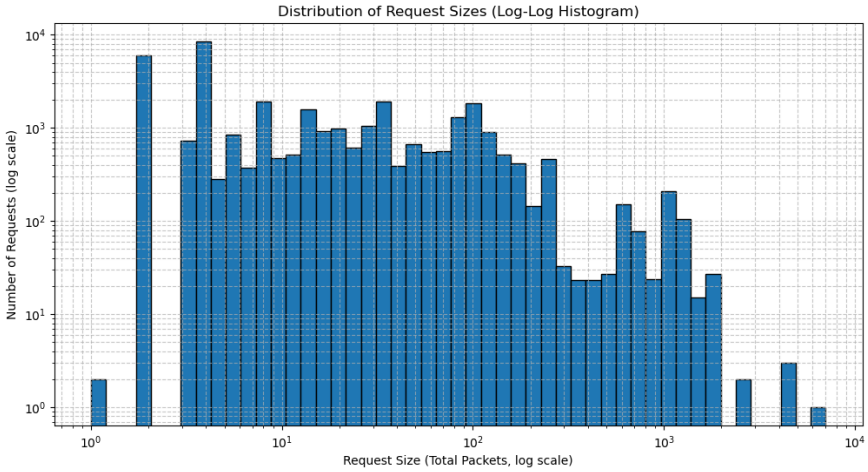
The outputs from the static and sequence branches are merged into a concatenated representation. The concatenated vector is passed through a dense layer (with 64 neurons and ReLU activation) along with an additional dropout layer with a 0.2 rate. Finally, a dense output layer with neurons equal to the number of attack classes (with SoftMax activation) produces a probability distribution when classifying instances (Goodfellow, Bengio and Courville, 2016), which is helpful when dealing with novel attacks.

5.4.2 RNN Implementation

As discussed, our initial RNN creation has two main data sources: a CSV file containing request metadata (static features) and a second CSV file containing packet-level data (sequential features), grouped by corresponding request IDs. Our preprocessing pipeline is as follows.

Static features are pre-processed by scaling numerical data using a standard scaler for

consistent value ranges, and categorical features are transformed using one-hot encoding. Processed features are stacked horizontally to form the final static feature matrix. As for sequential features, we apply one-hot encoding to protocol values. Given that packet sequences vary in length, we use TensorFlow's *pad_sequences* utility to pad sequences to a global maximum (capped at 1000 packets) to ensure a uniform input shape for the DNN. We cap the global limit at 1000 due to our distribution of packet quantity and our own computational limitations.



(Figure 5: Distribution of Request Sizes)

As shown in the above histogram, very few requests have over 1000 packets, and thus, to save on computational overhead, we can truncate extreme values with minimal effect on training.

Following this, we have a dedicated data loader class, *RequestSequenceLoader*, which manages dataset batching and constructs paired inputs for the static and sequence data. First, it verifies feature dimensions after processing and optimises the data handling using TensorFlow's *tf.data* pipeline. Caching stores pre-processed data so subsequent training epochs don't need to recompute expensive operations. Prefetching prepares the next batch of data as the current batch is being processed, minimising waiting time during training and helping to satisfy our requirement for a lightweight DNN. (TensorFlow, 2025)

A particularly crucial part of our sequential processing is the custom tokenizer implemented within *RequestSequenceLoader*. The tokenizer is designed to process strings of packet information and, later, the additional features we are extracting with GDB. The custom tokenizer's main functions include:

- Lowercasing all input text for uniformity

- Hexadecimal token handling. By using a regular expression of *0x[0-9a-f]+* and treating hexadecimals as single tokens, which is particularly important for when we start adding things like memory addresses into our input vector post-GDB augmentation.
- We split text on common delimiters (such as /, =, :, (,), ,) while retaining these delimiters as separate tokens. This approach preserves structural information in the info fields of our packets

Each token is then mapped to a pre-trained FastText embedding (Bojanowski et al., 2017). A zero vector is used as a fallback for tokens not found in the model. The resulting embeddings are later averaged (after padding) to produce a fixed-length representation that is concatenated with other packet-level features.

The network is compiled with the Adam optimiser and uses categorical cross-entropy as the loss function. At this point, we start tracking our evaluation metrics (accuracy, precision, recall and F1 score). The dataset is split using stratification based on attack categories, and after training, we can start evaluating and iteratively improving our DNN. Additionally, we keep the 6 least populated classes separate from the training dataset so that we can keep a log of these as "novel" attacks during evaluation in order to assess our model's ability to detect novel attacks, as well as assess how it classifies them.

Our secondary DNN builds upon the architecture of our primary model by incorporating the additional features extracted via GDB. The overall structure of the network remains largely identical to the primary RNN, using the same handling processes. The key difference is in the input. The goal is for the secondary RNN to become better equipped to learn subtle variations in the internal state indicative of benign versus malicious traffic.

5.5 Iterative Improvement

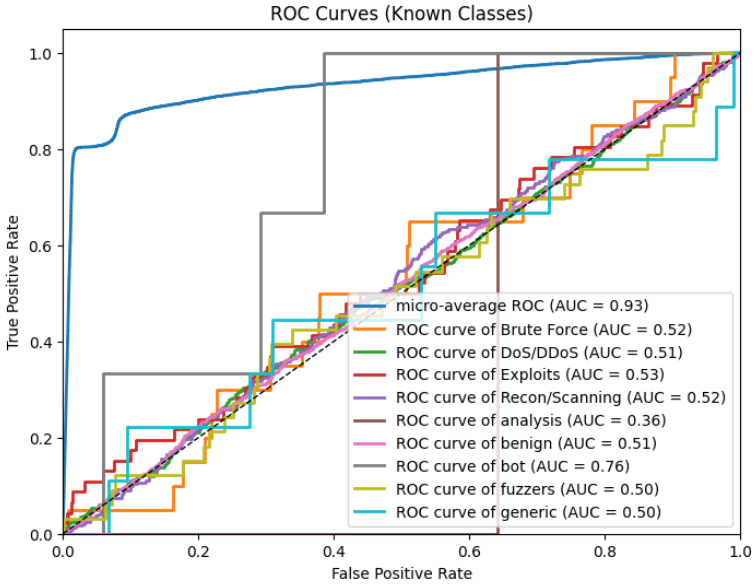
The experimentation process for refining the RNNs followed generally standard iterative improvement techniques and addressed some of the major imbalances present in our data.

The following steps outline our overall improvement methodology:

1. We begin by establishing a baseline RNN with our initial architecture, giving us a reference point to measure improvements.
2. Following this came hyperparameter tuning, including adjustments to the learning rate, dropout settings, number of units in the GRU layers, batch size and sequence length.
3. Architectural changes and preprocessing changes to address class imbalance were

experimented with, including weighted cross-entropy, SMOTE and focal loss. Our evaluation pipeline includes stratified splits of the dataset, ensuring a balanced representation of attack classes while ensuring that our test dataset will contain at least some examples of what the RNN would perceive as novel attacks.

Our initial model had little to account for the extreme class imbalance present in our dataset, and so while overall accuracy was high, it is here that we can directly see the false-positive fallacy rearing its' head; if our model is not encouraged to make "riskier" predictions for traffic, a high quantity of malicious traffic can be classified as benign since the model will learn that guessing benign more often than not will yield an overall high accuracy, as seen in the below ROC curve for our baseline model before any iterative improvement:



(Figure 6: ROC Curve of initial Model)

This tells us two things: first, overall accuracy is not a good metric for evaluating the effectiveness of our model, and second, we need to encourage the model to make harder decisions for underrepresented classes more often. To this end, we begin with an implementation of weighted cross-entropy. Weighted cross-entropy is an altered version of our initial loss function that applies weights to underrepresented classes, which forces the model to pay more attention to minority classes during training. While this approach certainly helped, alone it was not enough, and too many false negatives were occurring.

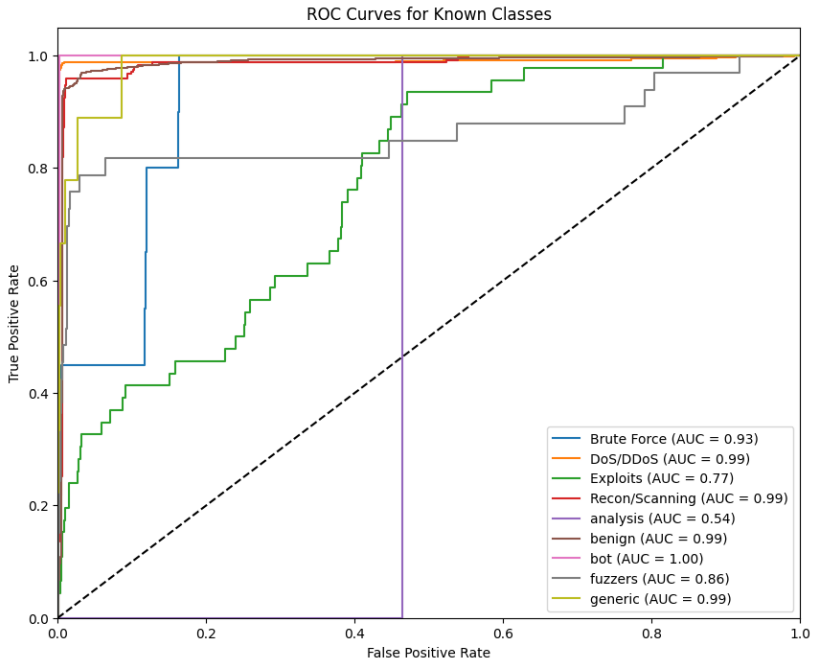
Following this, we add SMOTE (Synthetic Minority Oversampling Technique) to create artificial samples of minority classes. These synthetic samples are generated using an

algorithm similar to k-Nearest Neighbors, which calculates Euclidean distances between classes' instances and generates new values by interpolating neighbour values. SMOTE and weighted cross-entropy combined were effective, but weighted cross-entropy was creating too many false positives when combined with SMOTE since the weighting of classes does not apply to synthetic entries generated with SMOTE; thus, after further experimentation, we swapped to an implementation of focal loss. Focal loss is designed to address class imbalances by focusing on “hard” examples by weighing down the contribution of “easy” examples. Focal loss is a modification of cross-entropy, using a modulating factor that adjusts the loss function dynamically during training to prioritise underrepresented classes.

Lastly, after addressing the class imbalance, iterative testing with hyperparameter values yielded the following changes:

- Decreased learning rate to 0.001
- Increased secondary GRU layer from 32 to 46 neurons

The resulting ROC curve for our DNN model using SMOTE and focal loss with our hyperparameter tuning is as follows:



(Figure 7:ROC Curve of Baseline Model)

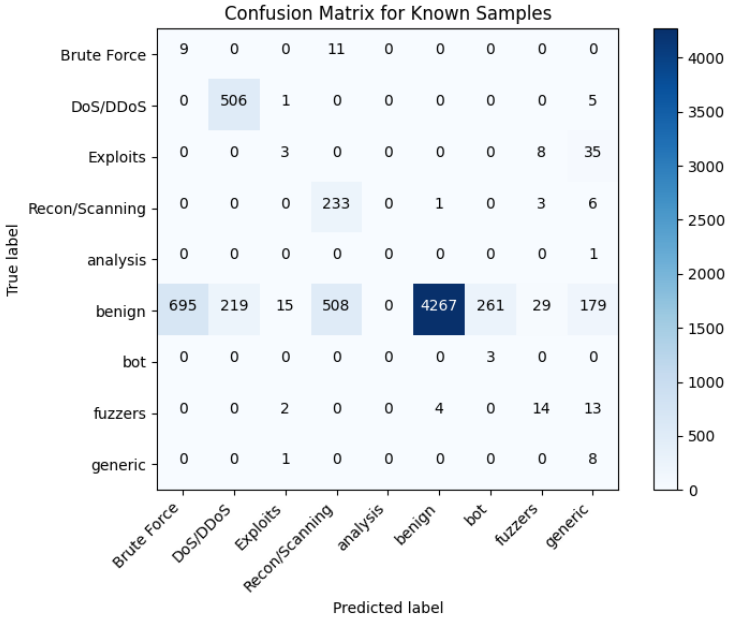
This process yielded two incarnations of our RNN, which are in Appendix 6. Now, we can

discuss and evaluate these models.

6 EVALUATION

6.1 Baseline RNN

Confusion Matrix:



(Figure 8:Confusion Matrix for Baseline Model)

(Table 1: Classification Report for Baseline Model)

	Precision	Recall	F1-score	Support
Brute Force	0.01	0.45	0.02	20
DoS/DDoS	0.70	0.99	0.82	512
Exploits	0.14	0.07	0.09	46
Recon/Scanning	0.31	0.96	0.47	243
Analysis	0.00	0.00	0.00	1
Benign	1.00	0.69	0.82	6173
Bot	0.01	1.00	0.02	3
Fuzzers	0.26	0.42	0.32	33
Generic	0.03	0.89	0.06	9
Accuracy			0.72	7040
Macro avg	0.27	0.61	0.29	7040

Weighted avg	0.94	0.72	0.79	7040

The unaugmented model achieves an overall accuracy of about 74% with a weighted F1-score of 0.79. As is consistent with IDS expectations of a traditional model, our confusion matrix shows a high false-positive rate, due to our need to compensate for the disproportionate representation of malicious traffic.

Performance is dominated by the benign class, which, as expected, is overwhelmingly overrepresented. The benign class has a high precision and decent recall, as do some of the more populated attack classes, such as Dos/DDoS and Recon/Scanning. This could be down to a number of reasons, other than pure class representation numbers; extreme packet loads characterise DDoS attacks in terms of length and quantity, and many extreme outliers in this metric can easily be classified as DDoS; the same applies to brute force attacks, which may explain why Brute Force has a relatively high recall, but not precision; many brute force attacks are likely being classified as DDoS due to overlapping metrics and being a less represented class. The extremely underrepresented classes such as analysis saw little to no predictions directly, which likely is due to the extreme underrepresentation of these classes..

In terms of novel attack detection, our unaugmented model performs well. It classified 17/18 instances of "novel" attacks as malicious, with only one being classified as benign, as seen below:

```
Unknown sample 1 predicted as: fuzzers
Unknown sample 2 predicted as: Exploits
Unknown sample 3 predicted as: generic
Unknown sample 4 predicted as: DoS/DDoS
Unknown sample 5 predicted as: DoS/DDoS
Unknown sample 6 predicted as: DoS/DDoS
Unknown sample 7 predicted as: DoS/DDoS
Unknown sample 8 predicted as: DoS/DDoS
Unknown sample 9 predicted as: DoS/DDoS
Unknown sample 10 predicted as: benign
Unknown sample 11 predicted as: DoS/DDoS
Unknown sample 12 predicted as: DoS/DDoS
Unknown sample 13 predicted as: DoS/DDoS
Unknown sample 14 predicted as: DoS/DDoS
Unknown sample 15 predicted as: DoS/DDoS
Unknown sample 16 predicted as: DoS/DDoS
Unknown sample 17 predicted as: DoS/DDoS
Unknown sample 18 predicted as: DoS/DDoS
```

(Figure 9: Novel Attack Detection for Baseline Model)

The vast majority of novel attacks were classified as Dos/DDoS, which could be due to DDoS being the most represented class that is not benign. Under our weightings applied by focal loss, it is possible that DDoS has become a sort of "fallback" option when it comes to potential threats.

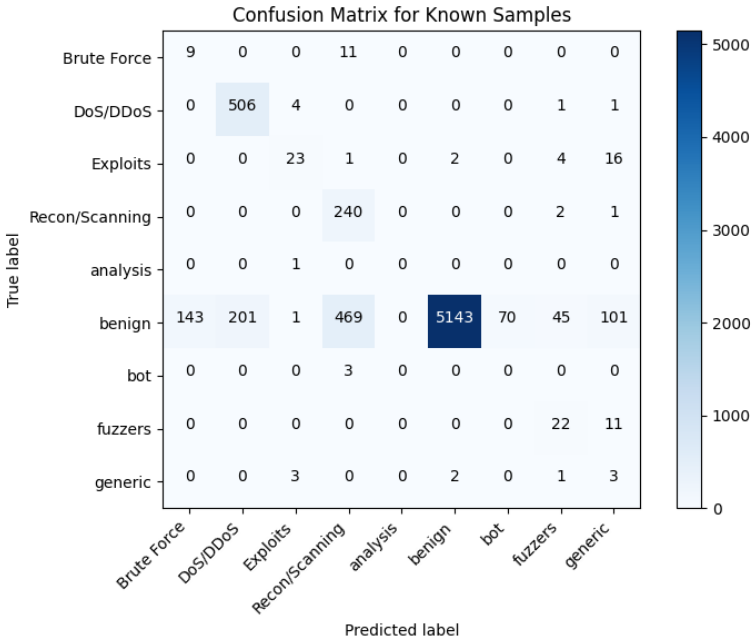
In a traditional IDS, it is more important to minimise false negatives than false positives and a high recall for attack classes is generally better. Our model, however, misses many attack instances (evidenced by very low recall scores on several attack categories), which is not acceptable for a security application. There are a few possible reasons for this:

- As mentioned previously, the overwhelming data imbalance. Even after adding focal loss and SMOTE, this imbalance may not be fully overcome, leading to the poor detection of rare attack types.
- Our implementation aggregates token embeddings (via averaging) for the packet "Info" field. While averaging is computationally efficient, which is necessary for our specific environmental use-case, it might lose some nuanced information for distinguishing subtle attack patterns, such as the difference between Brute Force and DDoS.

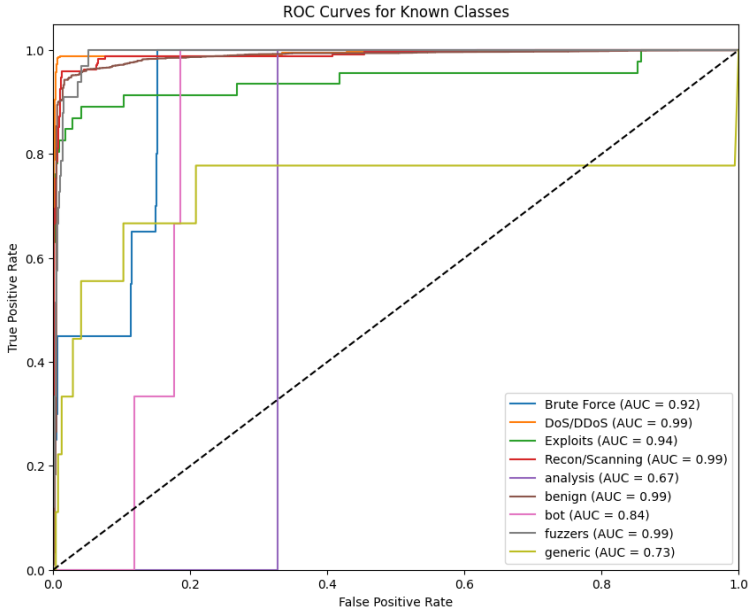
That said, for the purposes of our study, this is acceptable; we are not attempting to create the most accurate IDS possible but rather assess and measure the addition of GDB-augmented features into the input vector in terms of making up for the classic shortcomings of IDS: the false positive fallacy and novel attack detection inability. To that end, our augmented DNN yields the following results.

6.2 GDB-Augmented RNN

Confusion Matrix and ROC Curve:



(Figure 10: Confusion Matrix for Augmented RNN)



(Figure 11: ROC Curve of Augmented Model)

(Table 2: Classification Report for Augmented Model)

	Precision	Recall	F1-score	Support
Brute Force	0.06	0.45	0.10	20
DoS/DDoS	0.72	0.99	0.83	512
Exploits	0.72	0.50	0.59	46
Recon/Scanning	0.33	0.99	0.50	243
Analysis	0.00	0.00	0.00	1
Benign	1.00	0.83	0.91	6173
Bot	0.00	0.00	0.00	3
Fuzzers	0.29	0.67	0.41	33
Generic	0.02	0.33	0.04	9
Accuracy			0.84	7040
Macro avg	0.35	0.53	0.38	7040
Weighted avg	0.95	0.84	0.88	7040

As shown above, the introduction of our GDB-augmented features into our RNN model

demonstrably improved overall performance, as well as performance in our key area of interest: the false positive rate. Starting with our classification report, the augmented RNN achieved an accuracy of 84%, which is a notable increase from the unaugmented model's 72%. Similarly, the weighted F1-score rose from 0.79 to 0.88, which is indicative of a more balanced and effective model. Benign traffic saw a substantial improvement in recall, rising from 0.69 to 0.83, which suggests that our GDB features helped the model distinguish benign from malicious traffic, reducing the false positive rate; this is further evidenced by our confusion matrix, showing a decrease in false positives for DDoS, brute force, exploits, recon/scanning and fuzzers. Interestingly, however, our augmented model actually saw an increase in false positives for bot and generic classes. In addition to this, GDB augmentation had no effect on novel attack detection, producing the same classification output as our unaugmented model.

The model's ability to detect exploits improved significantly, with precision jumping from 0.14 to 0.72 and recall increasing from 0.07 to 0.50. This improvement could likely be due to the attack vectors that exploits operate on; causing disruptions in system states by exploiting vulnerable methods is absolutely something that would show up in GDB-augmented features, passing erroneous extreme values as arguments or abnormal memory regions. Fuzzers saw smaller, yet consistent improvements in all metrics. The model also retained high performance for our two dominant attack classes, DoS/DDoS and Recon/Scanning. Macro average F1-score increased from 0.29 to 0.38, indicating a general improvement in the model's ability to handle all classes, though there is still plenty of room to improve in terms of standard IDS capabilities. Specifically, it is likely that an overall increase in recall for benign data is due to consistency; much of our benign data has somewhat unchanging values in memory states, or variation from packet to packet remains consistent, possibly giving our RNN a better understanding of packet sequences for benign traffic. Conversely, it is possible that for generic attacks, the increase in information from our augmentation highlights more standard deviations from regular values, which may still be within expectations for benign traffic, but due to our sample size and the inherent ambiguity of the generic class, the model begins to predict generic more often. Similarly, bot attacks are typically characterized by periodic and automated behavioural patterns, which is likely similar to what you would expect from benign traffic regarding the specific features we extract with GDB, hence the increase in false positive rate.

7 DISCUSSION AND CONCLUSIONS

Our approach to incorporating an augmented input vector with GDB-derived features has proven effective in addressing shortcomings of traditional IDS; in major attack classes, the false-positive fallacy saw notable improvement, although there was no meaningful change in novel attack detection.

However, several limitations to our approach must be acknowledged. Firstly, due to the computational limitations posed by our environment, several techniques, such as global sequence length padding and averaging token embeddings for packet information and GDB features, while necessary for the challenges of a satellite environment, may have diluted temporal and sequential dependencies within packet sequences, potentially including subtle information for distinguishing between attacks not present in our current implementation. Future iterations should explore memory-efficient methods for aggregating sequential information, such as more efficient attention mechanisms or compression techniques.

Second, our simulation environment and breakpoint placement partially restricted our GDB augmentation. For the same reasons discussed in Chapter 5.4, the lack of variation in certain augmented features due to our focus on a point in the environment responsible for routing rather than payload execution could indicate the potential for further feature extraction, potentially capturing relations not encapsulated within our augmentations.

Lastly, class imbalance. Despite extensive mitigative strategies such as focal loss, weighted cross-entropy and SMOTE, it remains an inherent challenge for IDS and us as a whole. The low representation of certain attack classes inherently limits the model's generalization capabilities. By using more domain-specific synthetic data generation methods or alternative oversampling strategies to SMOTE could improve generalization ability. However, within the scope of this project, optimal IDS performance is not necessary to draw conclusions and evaluate our implementation.

In conclusion, whilst acknowledging these limitations, our methodological framework and implementation demonstrably shows promise for enhancing traditional IDS with our new method of deep packet analysis, warranting further investigation in a variety of domains and refinement of our augmentation process.

REFERENCES

Tanveer, H.B. et al. (2023) Making sense of constellations: Methodologies for understanding Starlink's scheduling algorithms, arXiv.org. Available at: https://arxiv.org/abs/2307.00402 (Accessed: 21 October 2024)

Heady, R., Luger, G. F., Maccabe, A., & Servilla, M. (1990). The architecture of a network-level intrusion detection system. https://doi.org/10.2172/425295.

Pachghare, V., Khatavkar, V., & Kulkarni, P. (2012). Pattern-based network security using semi-supervised learning. International Journal of Information and Network Security (Ijins), 1(3). https://doi.org/10.11591/ijins.v1i3.704

Yahi, S., Benferhat, S., & Kenaza, T. (2010). From using description logics to handling inconsistency in cooperative intrusion detection. 2010 International Conference on Machine and Web Intelligence. https://doi.org/10.1109/icmwi.2010.5648177

Giuliari, G., Ciussani, T., Perrig, A., Singla, A. and Zurich, E. (2021). ICARUS: Attacking low Earth orbit satellite networks ICARUS: Attacking low Earth orbit satellite networks.

Fratty, R., Saar, Y., Kumar, R. and Arnon, S. (2023). Random Routing Algorithm for Enhancing the Cybersecurity of LEO Satellite Networks. Electronics, 12(3), p.518. doi:https://doi.org/10.3390/electronics12030518.

Izhikevich, L., Tran, M., Izhikevich, K., Akiwate, G. and Durumeric, Z. (2024). Democratising LEO Satellite Network Measurement. ACM SIGMETRICS Performance Evaluation Review, 52(1), pp.15–16. doi:https://doi.org/10.1145/3673660.3655052.

Elsadai, A., Ibrahim, J., Hajjaj, F., & Jakić, P. (2019). The overview of intrusion detection system methods and techniques. Proceedings of the International Scientific Conference - Sinteza 2019, 155-161. https://doi.org/10.15308/sinteza-2019-155-161.

Tejakshi, N. S., Vyshnavi, M. K., & Manjuprasad, B. (2018). Measuring of data quality in kyc using anomaly detection techniques. Nciccnda. https://doi.org/10.21467/proceedings.1.39

Liu, H. and Liu, B. (2019). Machine learning and deep learning methods for intrusion detection systems: a survey. Applied Sciences, 9(20), 4396.

https://doi.org/10.3390/app9204396

Zhong, W., Yu, N., & Ai, C. (2020). Applying big data based deep learning system to intrusion detection. Big Data Mining and Analytics, 3(3), 181-195. https://doi.org/10.26599/bdma.2020.9020003

Smailes, J., Köhler, S., Birnbach, S., Strohmeier, M., & Martinovic, I. (2023). Watch this space: securing satellite communication through resilient transmitter fingerprinting. Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security. https://doi.org/10.1145/3576915.3623135

Liu, Y., Feng, L., Wu, L., Zhang, Z., Dang, J., Zhu, B., ... & Wang, L. (2021). Joint optimization based satellite handover strategy for low earth orbit satellite networks. IET Communications, 15(12), 1576-1585. https://doi.org/10.1049/cmu2.12170

Park, S. and Kim, J. (2021). Trends in leo satellite handover algorithms.. https://doi.org/10.48550/arxiv.2107.08619

Li, G., Sun, H., Zhao, X., & Wang, L. (2022). A distributed intrusion detection system based on cnn-gru in cloud environment. International Conference on Artificial Intelligence and Intelligent Information Processing (AIIIP 2022). https://doi.org/10.1117/12.2660060

Khan, W. U., Ali, Z., Lagunas, E., Chatzinotas, S., & Ottersten, B. (2022). Rate splitting multiple access for cognitive radio geo-leo co-existing satellite networks.. https://doi.org/10.48550/arxiv.2208.02924

Al-Hraishawi, H., Chougrani, H., Kisseleff, S., Lagunas, E., & Chatzinotas, S. (2021). A survey on non-geostationary satellite systems: the communication perspective.. https://doi.org/10.48550/arxiv.2107.05312

Chen, R., Wang, W., Zhao, X., & Zhao, G. (2022). Waypoint segment routing algorithm for leo satellite network. IET Communications, 16(18), 2133-2144. https://doi.org/10.1049/cmu2.12466

Oligeri, G., Raponi, S., Sciancalepore, S., & Pietro, R. D. (2023). Past-ai: physical-layer authentication of satellite transmitters via deep learning. IEEE Transactions on Information Forensics and Security, 18, 274-289. https://doi.org/10.1109/tifs.2022.3219287

Zhang, Y., Wang, Y., Hu, Y., Lin, Z., Zhai, Y., Wang, L., ... & Kang, L. (2022). Security performance analysis of leo satellite constellation networks under ddos attack

Sensors, 22(19), 7286. https://doi.org/10.3390/s22197286
IBM (2023). What are Recurrent Neural Networks? | IBM. [online] www.ibm.com. Available at: https://www.ibm.com/topics/recurrent-neural-networks

Ahmad, N., Herdiwijaya, D., Djamaluddin, T., & Usui, H. (2018). Diagnosing low earth orbit satellite anomalies using noaa-15 electron data associated with geomagnetic perturbations. Earth, Planets and Space, 70(1) https://doi.org/10.1186/s40623-018-0852-2

Vasylyev, D., Vogel, W., & Moll, F. (2019). Satellite-mediated quantum atmospheric links. Physical Review A, 99(5). https://doi.org/10.1103/physreva.99.053830

Guoyan, L., Tian, L., Yue, X., Hou, T., & Dai, B. (2023). High reliable uplink transmission methods in geo–leo heterogeneous satellite network. Applied Sciences, 13(15), 8611. https://doi.org/10.3390/app13158611

Wrona, A. and Tantucci, A. (2023). Artificial intelligence--based data path control in leo satellites--driven optical communications.. https://doi.org/10.22541/au.169744685.51020680/v1

Raza, M. (2018). What is the OSI Model? Explore the 7 Layers of the Open Systems Interconnection Model. [online] BMC Blogs. Available at: https://www.bmc.com/blogs/osi-model-7-layers/

Raja, M. A. Z., Güven, U., & Prakash, O. (2020). Design and analysis of attitude control algorithm for low earth orbiting satellite with magnetic torquer concepts using nonlinear unscented kalman filter. International Journal of Engineering &Amp; Technology, 9(1), 37-51. https://doi.org/10.14419/ijet.v7i2.17.10079

He, M., Cui, G., Wu, M., & Wang, W. (2024). Collaborative interference avoidance technology in geo-leo co-existing satellite system. International Journal of Satellite Communications and Networking, 42(4), 257-272. https://doi.org/10.1002/sat.1511

Leyva-Mayorga, I., Soret, B., Röper, M., Wübben, D., Matthiesen, B., Dekorsy, A., ... & Popovski, P. (2020). Leo small-satellite constellations for 5g and beyond-5g

communications. IEEE Access, 8, 184955-184964. https://doi.org/10.1109/access.2020.3029620

Shen, J., Wang, D., Wang, J., Zang, P., Shi, X., & Wang, Y. (2022). Time-sensitive satellite internet based on uniform content labels. Journal of Web Engineering. https://doi.org/10.13052/jwe1540-9589.21217

Rani, M. A. U. (2017). Analysis of inter-satellite optical wireless channel for improved transmission and data rate. International Journal for Research in Applied Science and Engineering Technology, V(X), 2189-2196. https://doi.org/10.22214/ijraset.2017.10322

Chaudhry, A.U. and Yanikomeroglu, H. (2021). Laser Intersatellite Links in a Starlink Constellation: A Classification and Analysis. *IEEE Vehicular Technology Magazine*, [online] 16(2), pp.48–56. doi:https://doi.org/10.1109/MVT.2021.3063706

Lü, Y., Sun, F., Zhao, Y., Li, H., & Liu, H. (2013). Distributed traffic balancing routing for leo satellite networks. International Journal of Computer Network and Information Security, 6(1), 19-25. https://doi.org/10.5815/ijcnis.2014.01.03

Sanctis, M. D., Cianca, E., Araniti, G., Bisio, I., & Prasad, R. (2016). Satellite communications supporting internet of remote things. IEEE Internet of Things Journal, 3(1), 113-123. https://doi.org/10.1109/jiot.2015.2487046

CCSDS Historical Document. (n.d.). Available at: https://public.ccsds.org/Pubs/133x0b1s.pdf [Accessed 30 Oct. 2024].

Salkield, E., Köhler, S., Birnbach, S., Baker, R., Strohmeier, M., & Martinovic, I. (2023). Firefly: spoofing earth observation satellite data through radio overshadowing. Proceedings 2023 Workshop on Security of Space and Satellite Systems. https://doi.org/10.14722/spacesec.2023.231879

Foust, J. and Berger, B. (2023) SpaceX shifts resources to cybersecurity to address Starlink Jamming, SpaceNews. Available at: https://spacenews.com/spacex-shifts-resources-to-cybersecurity-to-address-starlink-jamming/ (Accessed: 31 October 2024).

Yang, Y., Zheng, K., Wu, B., Yang, Y., & Wang, X. (2020). Network intrusion detection based on supervised adversarial variational auto-encoder with regularization. IEEE

Access, 8, 42169-42184. https://doi.org/10.1109/access.2020.2977007

Nguyen, X., Nguyen, X., & Le, K. (2022). Realguard: a lightweight network intrusion detection system for iot gateways. Sensors, 22(2), 432. https://doi.org/10.3390/s22020432

K. Bhardwaj, "Understanding the FTP Attack and Defense Techniques," International Journal of Computer Applications, 2019

M. F. Khoshbakht et al., "SSH Brute Force Attack Detection Based on Machine Learning Algorithms," Journal of Information Security and Applications, 2021.

M. S. Thaduri et al., "Denial of Service (DoS) Attack: A Review," International Journal of Computer Applications, 2020

K. J. Sharma and N. K. Sharma, "Denial of Service Attacks: A Comprehensive Review," International Journal of Advanced Research in Computer Science, 2018
R., I. (2021) DDoS attack using slowhttptest (slowloris) in Kali Linux, Medium. Available at: https://techinnovatehub.medium.com/ddos-attack-using-slowhttptest-slowloris-in-kali-linux-3bd26d9d991a (Accessed: 31 October 2024).

What is slowloris?: Examples & mitigation techniques: Imperva (2023) Learning Center. Available at: https://www.imperva.com/learn/ddos/slowloris/ (Accessed: 31 October 2024).

Synopsys, Inc. http://www.synopsys.com/ (no date) The heartbleed bug, Heartbleed Bug. Available at: https://heartbleed.com/ (Accessed: 31 October 2024).

What is a brute force attack?: Definition, Types & How It Works (no date) Fortinet. Available at: https://www.fortinet.com/resources/cyberglossary/brute-force-attack (Accessed: 31 October 2024).

Abdullah, A., Muhammad and Malik, M. (2022) A survey on SQL injection attacks: Detection and prevention. Available at: https://www.researchgate.net/publication/361444044_A_Survey_on_SQL_Injection_Attacks_Detection_and_Prevention (Accessed: 31 October 2024).

What is infiltration? (no date) What is Infiltration? - Cybersecurity Breaches. Available at: https://cyberpedia.reasonlabs.com/EN/infiltration.html (Accessed: 31

October 2024).

Muhammad Ijaz Ul Haq (2021) (PDF) survey of Port Scanning Detection Techniques. 26
Available at:
https://www.researchgate.net/publication/356782133_Survey_of_Port_Scanning_Detection_Techniques (Accessed: 31 October 2024).

IMPROVING STARLINK'S LATENCY. (n.d.). Available at: https://www.starlink.com/public-files/StarlinkLatency.pdf.

Wainscott-Sargent, A. (2022). Satellite Operators Respond to Cyber Threats in a Rapidly Changing Environment. [online] Satellitetoday.com. Available at: https://interactive.satellitetoday.com/via/october-2022/satellite-operators-respond-to-cyber-threats-in-a-rapidly-changing-environment [Accessed 18 Nov. 2024].

What is a ddos attack? ddos meaning, definition & types (no date) Fortinet. Available at: https://www.fortinet.com/resources/cyberglossary/ddos-attack#:~:text=DDoS%20Attack%20means%20"Distributed%20Denial,connected%20online%20services%20and%20sites. (Accessed: 31 October 2024).

Khraisat, A., Gondal, I., Vamplew, P. and Kamruzzaman, J. (2019). Survey of intrusion detection systems: techniques, datasets and challenges. Cybersecurity, [online] 2(1), pp.1–22. doi:https://doi.org/10.1186/s42400-019-0038-7.

Ryu, Y. U. and Rhee, H. (2008). Evaluation of intrusion detection systems under a resource constraint. ACM Transactions on Information and System Security, 11(4), 1-24. https://doi.org/10.1145/1380564.1380566

Cuppens, F., Gombault, S., & Sans, T. Selecting appropriate counter-measures in an intrusion detection framework. Proceedings. 17th IEEE Computer Security Foundations Workshop, 2004., 78-87. https://doi.org/10.1109/csfw.2004.1310733

Axelsson, S. (2000). The base-rate fallacy and the difficulty of intrusion detection. ACM Transactions on Information and System Security, 3(3), 186-205. https://doi.org/10.1145/357830.357849

Weerakody, P. B., Wong, K. W., & Wang, G. (2022). Cyclic gate recurrent neural networks for time series data with missing values. Neural Processing Letters, 55(2),

1527-1554. https://doi.org/10.1007/s11063-022-10950-2

Bravo, J. M. (2021). Forecasting longevity for financial applications: a first experiment with deep learning methods. Communications in Computer and Information Science, 232-249. https://doi.org/10.1007/978-3-030-93733-1_17

Tang, T. A., Mhamdi, L., McLernon, D., Zaidi, S. A. R., & Ghogho, M. (2016). Deep learning approach for network intrusion detection in software defined networking. 2016 International Conference on Wireless Networks and Mobile Communications (WINCOM). https://doi.org/10.1109/wincom.2016.7777224

Shah, D. (2023). Cross Entropy Loss: Intro, Applications, Code. [online] www.v7labs.com. Available at: https://www.v7labs.com/blog/cross-entropy-loss-guide

Shastri, Y. (2023). A Step-by-Step Guide to Federated Learning in Computer Vision. [online] www.v7labs.com. Available at: https://www.v7labs.com/blog/federated-learning-guide

Free Software Foundation (2024). Top (Debugging with GDB). [online] Sourceware.org. Available at: https://sourceware.org/gdb/current/onlinedocs/gdb.html/Breakpoints.html#Breakpoints
[Accessed 17 Nov. 2024].

Free Software Foundation (2024). Top (Debugging with GDB). [online] Sourceware.org. Available at: https://sourceware.org/gdb/current/onlinedocs/gdb.html/C.html#C [Accessed 17 Nov. 2024].

Free Software Foundation (2024). Top (Debugging with GDB). [online] Sourceware.org. Available at: https://sourceware.org/gdb/current/onlinedocs/gdb.html/Registers.html#Registers [Accessed 17 Nov. 2024].

Free Software Foundation (2024). Top (Debugging with GDB). [online] Sourceware.org. Available at: https://sourceware.org/gdb/current/onlinedocs/gdb.html/Floating-Point-Hardware.html#Floating-Point-Hardware
[Accessed 17 Nov. 2024].

Sourceware.org. (2025). *Breakpoints (Debugging with GDB)*. [online] Available at: https://sourceware.org/gdb/current/onlinedocs/gdb.html/Breakpoints.html#Breakpoints [Accessed 23 Mar. 2025].

Sourceware.org. (2025). *sourceware.org Git - binutils-gdb.git/summary*. [online] Available at: https://sourceware.org/git/?p=binutils-gdb.git [Accessed 23 Mar. 2025].
Docker Documentation. (2024). *Running containers*. [online] Available at: https://docs.docker.com/engine/containers/run/#runtime-privilege-and-linux-capabilities [Accessed 23 Mar. 2025].

Team, F.S. (2020). *Cybersecurity Glossary | Courtesy of Fortify24x7 | Cyberthreat Blog from Fortify24x7*. [online] Cyberthreat.blog. Available at: https://cyberthreat.blog/cybersecurity-glossary/ [Accessed 24 Mar. 2025].

Miller, B.P., Fredriksen, L. and So, B. (1990). An empirical study of the reliability of UNIX utilities. *Communications of the ACM*, 33(12), pp.32–44. doi:https://doi.org/10.1145/96267.96279.

Kocher, P., Jaffe, J. and Jun, B. (1999). Differential Power Analysis. *Advances in Cryptology — CRYPTO' 99*, pp.388–397. doi:https://doi.org/10.1007/3-540-48405-1_25.

Schneier, B. (2024). *Cloudflare Reports that Almost 7% of All Internet Traffic Is Malicious - Schneier on Security*. [online] Schneier on Security. Available at: https://www.schneier.com/blog/archives/2024/07/cloudflare-reports-that-almost-7-of-all-internet-traffic-is-malicious.html [Accessed 26 Mar. 2025].

wiki.wireshark.org. (n.d.). *Ethernet*. [online] Available at: https://wiki.wireshark.org/Ethernet.

wiki.wireshark.org. (n.d.). *Internet_Protocol*. [online] Available at: https://wiki.wireshark.org/Internet_Protocol.

Wireshark.org. (2019). *Wireshark · Display Filter Reference: Transmission Control Protocol*. [online] Available at: https://www.wireshark.org/docs/dfref/t/tcp.html.

Wireshark. (2025). *Wireshark · Display Filter Reference: Secure Sockets Layer*. [online] Available at: https://www.wireshark.org/docs/dfref/s/ssl.html [Accessed 23 Mar. 2025].

Wireshark. (2025). *Wireshark · Display Filter Reference: Transport Layer Security*. [online] Available at: https://www.wireshark.org/docs/dfref/t/tls.html [Accessed 23 Mar. 2025].

Liu, M., Xue, Z., Xu, X., Zhong, C. and Chen, J. (2018). Host-Based Intrusion Detection System with System Calls. *ACM Computing Surveys*, 51(5), pp.1–36. doi:https://doi.org/10.1145/3214304.

Silberschatz, A., Galvin, P.B. and Gagne, G. (2018). *Operating system concepts*. Hoboken, Nj: Wiley.

Kernighan, B.W. and Ritchie, D.M. (1988). *The C Programming Language*.

Sourceware.org. (2025). *Disassembly In Python (Debugging with GDB)*. [online] Available at: https://sourceware.org/gdb/current/onlinedocs/gdb.html/Disassembly-In-Python.html#Disassembly-In-Python [Accessed 23 Mar. 2025].

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L. and Polosukhin, I. (2017). *Attention Is All You Need*. [online] arXiv. Available at: https://arxiv.org/abs/1706.03762.

Bahdanau, D., Cho, K. and Bengio, Y. (2014). *Neural Machine Translation by Jointly Learning to Align and Translate*. [online] arXiv.org. Available at: https://arxiv.org/abs/1409.0473.

Goodfellow, I., Bengio, Y. and Courville, A. (2016). *Deep Learning*. [online] Cambridge, Massachusetts: The MIT Press. Available at: https://www.deeplearningbook.org/.

TensorFlow. (2025). *tf.data.Dataset | TensorFlow v2.16.1*. [online] Availble at: https://www.tensorflow.org/api_docs/python/tf/data/Dataset#as_numpy_iterator [Accessed 23 Mar. 2025].

Bojanowski, P., Grave, E., Joulin, A. and Mikolov, T. (2017). Enriching Word Vectors with Subword Information. *Transactions of the Association for Computational Linguistics*, 5, pp.135–146. doi:https://doi.org/10.1162/tacl_a_00051.

Dataset References

LENS

Below is the required reference to the LENS dataset and its authors, in the format it is listed as on the public github repository:

@inproceedings{10.1145/3625468.3652170,
author = {Zhao, Jinwei and Pan, Jianping},
28
title = {LENS: A LEO Satellite Network Measurement Dataset},
year = {2024},
isbn = {9798400704123},
publisher = {Association for Computing Machinery},
address = {New York, NY, USA},
url = {https://doi.org/10.1145/3625468.3652170},
doi = {10.1145/3625468.3652170},
booktitle = {Proceedings of the 15th ACM Multimedia Systems Conference},
pages = {278–284},
numpages = {7},
keywords = {Dataset, Inter-Satellite Links, LEO, Latency, Network Measurement},
location = {Bari, Italy},
series = {MMSys '24}
}

CIC-IDS2017

Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization", 4th International Conference on Information Systems Security and Privacy (ICISSP), Portugal, January 2018.

UNSW-NB15

Moustafa, Nour, and Jill Slay. "UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set)." *Military Communications and Information Systems Conference (MilCIS)*, 2015. IEEE, 2015.Moustafa, Nour, and Jill Slay. "The evaluation of Network Anomaly Detection Systems: Statistical analysis of the UNSW-NB15 dataset and the comparison with the KDD99 dataset." *Information Security Journal: A Global Perspective* (2016): 1-14.

Moustafa, Nour, et al. "Novel geometric area analysis technique for anomaly detection using trapezoidal area estimation on large-scale networks." *IEEE Transactions on Big Data (2017).*

Moustafa, Nour, et al. "Big data analytics for intrusion detection system: statistical decision-making using finite dirichlet mixture models." *Data Analytics and Decision Support for Cybersecurity. Springer, Cham, 2017.* 127-156.

Sarhan, Mohanad, Siamak Layeghy, Nour Moustafa, and Marius Portmann. NetFlow Datasets for Machine Learning-Based Network Intrusion Detection Systems. In *Big Data Technologies and Applications: 10th EAI International Conference, BDTA 2020, and 13th EAI International Conference on Wireless Internet, WiCON 2020, Virtual Event, December 11, 2020, Proceedings* (p. 117). Springer Nature.

Kitsune Network Attack Dataset
Below is the reference in the format it is presented in on the public Kaggle page:

@inproceedings{mirsky2018kitsune, title={Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection}, author={Mirsky, Yisroel and Doitshman, Tomer and Elovici, Yuval and Shabtai, Asaf}, booktitle={The Network and Distributed System Security Symposium (NDSS) 2018}, year={2018} }

License Acknowledgements

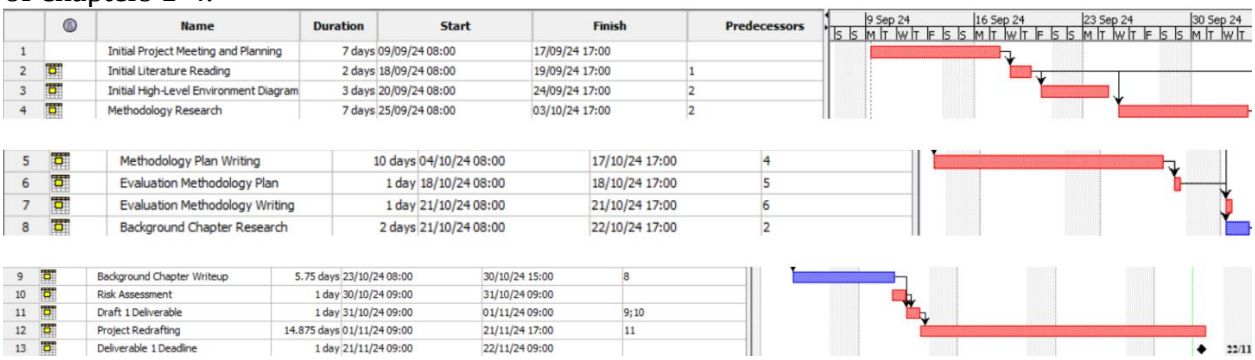
I hereby acknowledge and commit to all licensing, terms and conditions, nonprofit agreements and commercial usage licences and other stipulations and conditions for all technologies listed in this project, and provide a list of all license agreements as links below:

- OMNeT++: https://omnetpp.org/intro/license.html
- Wireshark: https://www.wireshark.org/docs/wsdg_html_chunked/AppGPL.html
- GDB: https://ftp.gnu.org/old-gnu/Manuals/gdb/html_chapter/gdb_1.html#:~:text=GDB%20is%20free%20software%2C%20protected,General%20Public%20License%20(GPL)
- GNS3: https://github.com/GNS3/gns3-vm/blob/master/LICENSE
- NetCat: https://github.com/diegocr/netcat/blob/master/license.txt
- SoCat: https://github.com/alpine-docker/socat/blob/master/LICENSE
- iPerf3: https://github.com/R0GGER/public-iperf3-servers/blob/main/LICENSE

A APPENDIX: Project Management

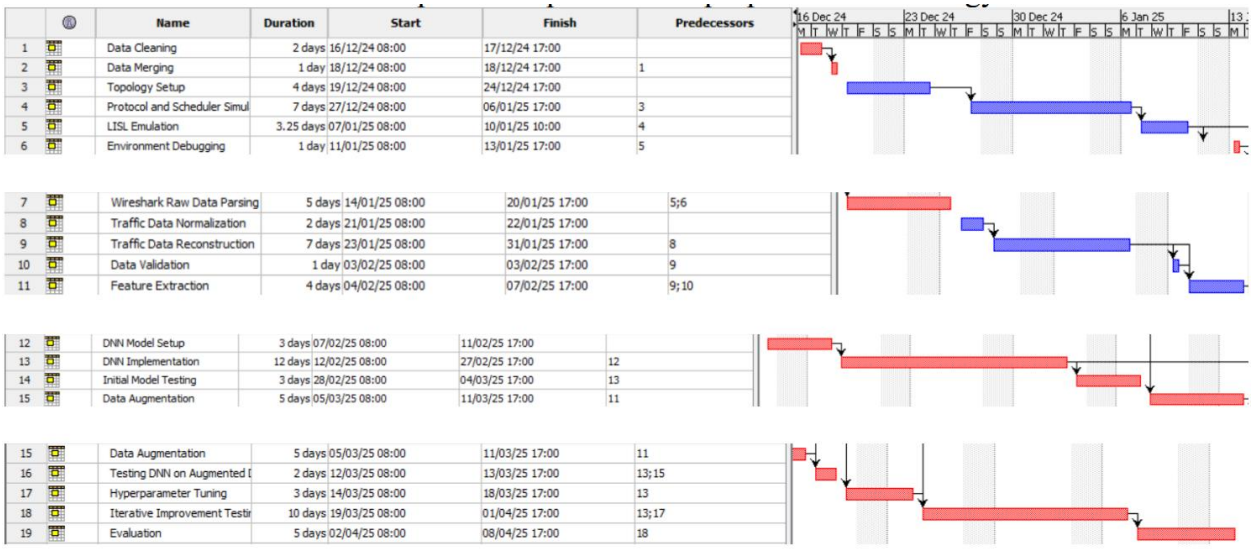
In this appendix, we will briefly cover the risk associated with the project and go over our timeline plan to implement the steps of the proposed methodology in the form of a Gantt chart. We will also review the writing of the front and back matter in the form of a Gantt chart.

Our first Gantt Chart is the front and back matter plan, covering the research and writing of chapters 1-4.



	Name	Duration	Start	Finish	Predecessors
1	Initial Project Meeting and Planning	7 days	09/09/24 08:00	17/09/24 17:00	
2	Initial Literature Reading	2 days	18/09/24 08:00	19/09/24 17:00	1
3	Initial High-Level Environment Diagram	3 days	20/09/24 08:00	24/09/24 17:00	2
4	Methodology Research	7 days	25/09/24 08:00	03/10/24 17:00	2
5	Methodology Plan Writing	10 days	04/10/24 08:00	17/10/24 17:00	4
6	Evaluation Methodology Plan	1 day	18/10/24 08:00	18/10/24 17:00	5
7	Evaluation Methodology Writing	1 day	21/10/24 08:00	21/10/24 17:00	6
8	Background Chapter Research	2 days	21/10/24 08:00	22/10/24 17:00	2
9	Background Chapter Writeup	5.75 days	23/10/24 08:00	30/10/24 15:00	8
10	Risk Assessment	1 day	30/10/24 09:00	31/10/24 09:00	
11	Draft 1 Deliverable	1 day	31/10/24 09:00	01/11/24 09:00	9;10
12	Project Redrafting	14.875 days	01/11/24 09:00	21/11/24 17:00	11
13	Deliverable 1 Deadline	1 day	21/11/24 09:00	22/11/24 09:00	

Our second Gantt Chart shows the implementation schedule of the proposed methodology



	Name	Duration	Start	Finish	Predecessors
1	Data Cleaning	2 days	16/12/24 08:00	17/12/24 17:00	
2	Data Merging	1 day	18/12/24 08:00	18/12/24 17:00	1
3	Topology Setup	4 days	19/12/24 08:00	24/12/24 17:00	
4	Protocol and Scheduler Simul	7 days	27/12/24 08:00	06/01/25 17:00	3
5	LISL Emulation	3.25 days	07/01/25 08:00	10/01/25 10:00	4
6	Environment Debugging	1 day	11/01/25 08:00	13/01/25 17:00	5
7	Wireshark Raw Data Parsing	5 days	14/01/25 08:00	20/01/25 17:00	5;6
8	Traffic Data Normalization	2 days	21/01/25 08:00	22/01/25 17:00	
9	Traffic Data Reconstruction	7 days	23/01/25 08:00	31/01/25 17:00	8
10	Data Validation	1 day	03/02/25 08:00	03/02/25 17:00	9
11	Feature Extraction	4 days	04/02/25 08:00	07/02/25 17:00	9;10
12	DNN Model Setup	3 days	07/02/25 08:00	11/02/25 17:00	
13	DNN Implementation	12 days	12/02/25 08:00	27/02/25 17:00	12
14	Initial Model Testing	3 days	28/02/25 08:00	04/03/25 17:00	13
15	Data Augmentation	5 days	05/03/25 08:00	11/03/25 17:00	11
15	Data Augmentation	5 days	05/03/25 08:00	11/03/25 17:00	11
16	Testing DNN on Augmented I	2 days	12/03/25 08:00	13/03/25 17:00	13;15
17	Hyperparameter Tuning	3 days	14/03/25 08:00	18/03/25 17:00	13
18	Iterative Improvement Testir	10 days	19/03/25 08:00	01/04/25 17:00	13;17
19	Evaluation	5 days	02/04/25 08:00	08/04/25 17:00	18

Below is a risk assessment table for the proposed methodology. The risk column identifies the risk, the probability is the likelihood of the risk occurring, the impact is the effect the risk will have on project development, mitigation is preemptive steps taken to avoid the risk, and action is the planned response if the risk occurs.

RISK	PROBABILITY	IMPACT	MITIGATION	ACTION
One or more datasets are unsuitable	Low	High	Early dataset analysis for suitability	Locate additional datasets and acquire ethical approval
Time crunch for GDB augmentation	Medium	High	Ensure augmentation process is done through scripts	Without sacrificing functionality, prioritise essential augmentations
Falling behind project schedule	Low	High	Follow created Gantt chart(s)	Reschedule less critical tasks to create backlog
Delays in DNN training because	Medium	Medium	Schedule training to be	Offload training to

of computational overhead			performed concurrently with other tasks	cloud resources
Any errors in GDB scripting would affect quality and realism	Medium	High	Iteratively increase the scale of GDB tests until scaling to the entire dataset	Adjust augmentation scripts based on DNN performance
Errors in data cleaning/merging.	Medium	Low	Create verification scripts for data cleaning and merging	Remove the errors and resume the task
Data loss during augmentation	Low	Medium	Regularly backup datasets	Restore dataset backups and resume

B APPENDIX: Professional, Legal, Ethical and Social Considerations

B.1 Professional Issues

This project will commit to all licensing terms of all datasets, libraries, software, models, and frameworks utilised. In developing our IDS, this project will also adhere to the Python Enhancement Proposal (PEP-8) to ensure readability and consistency.

B.2 Legal Issues

This project exclusively uses publicly available data, which complies with data privacy regulations such as GDPR. Although the data is from real-world sources, all personal information has been obfuscated. All ownerships or licences have also been acknowledged in a subsection of references titled license acknowledgement.

B.3 Ethical issues

Primarily, the ethical considerations for this project revolve around the responsible use of the proposed technology, given that it has the capability to analyse potentially personal information. While the IDS aims to protect against threats, it must be designed in such a way that, at all points, privacy is respected when analysing data and that any and all publicly available data is used in such a way that no individual's privacy is put at risk. Ethical diligence includes ensuring the system is not capable of being misused for

surveillance or other intrusive purposes beyond its intended objectives. To this end, we will ensure that while the system analyses data, it will have no capabilities to store that data, that its operating mechanisms are transparent, and that documentation is exhaustive.

B.4 Social Issues

Satellite networks are responsible for important parts of global communication, navigation, weather forecasting, and other vital infrastructures. Thus, we must aim to develop an IDS that secures only these vital resources. Additionally, it must not be a replacement for any human enforcement of laws or privacy regulations. We will try to distance ourselves from all of these issues through exhaustive obfuscation of any personal or private data during the project.

C APPENDIX: Dataset Construction Code

Below is the essential code for illustrating the process of our dataset construction.

C.1 Feature Extraction Code

The following code processes PCAP files and merges derived packet-level features with static metadata - This script does most of the heavy lifting in terms of dataset construction

Renames headers based on a predefined mapping and outputs two CSV files:

- A merged CSV with derived features per request
- A CSV with static metadata

Parameters:

- static_csv_path: Path to the primary CSV file with static network data.
- metadata_csv_path: Path to the CSV file with additional satellite metadata.
- pcap_directory: Directory containing per-request PCAP files.
- output_merged_csv: Output CSV file for the merged data.
- output_metadata_csv: Output CSV file for the static metadata.

Mapping of original column names to target names; fields mapped to 'N/A' are ignored

(Small subset of the header mapping)

```
header_mapping = {
    'Flow ID': 'N/A',
    'Source IP': 'srcip',
    'Source Port': 'sport',
    'Destination IP': 'dstip',
    'Destination Port': 'dsport',
    'Protocol': 'proto',
    'Timestamp': 'Stime',
    'Flow Duration': 'dur',
    'Total Fwd Packets': 'Spkts',
    'Total Backward Packets': 'Dpkts',
    'Total Length of Fwd Packets': 'sbytes',
    'Total Length of Bwd Packets': 'dbytes',
```

Following this, we load and dissect our PCAP files:

```
        for packet in packets:
            if IP in packet:
                timestamps.append(packet.time)
                packet_lengths.append(len(packet))
                src_ips.append(packet[IP].src)
                dst_ips.append(packet[IP].dst)
                ttls.append(packet[IP].ttl)

                #Determine direction: 1 if packet from source to destination, 0 for reverse
                if packet[IP].src == row['Source IP'] and packet[IP].dst == row['Destination IP']:
                    direction = 1
                elif packet[IP].src == row['Destination IP'] and packet[IP].dst == row['Source IP']:
                    direction = 0
                else:
                    direction = -1
                directions.append(direction)

                #Extract port and TCP flag information
                if TCP in packet:
                    src_ports.append(packet[TCP].sport)
                    dst_ports.append(packet[TCP].dport)
                    tcp_flags.append(packet[TCP].flags)
                elif UDP in packet:
                    src_ports.append(packet[UDP].sport)
                    dst_ports.append(packet[UDP].dport)
                    tcp_flags.append(0)
```

```
            else:
                src_ports.append(0)
                dst_ports.append(0)
                tcp_flags.append(0)
        else:
            #For non-IP packets, mark values accordingly
            timestamps.append(packet.time)
            packet_lengths.append(len(packet))
            directions.append(-1)
            src_ips.append(None)
            dst_ips.append(None)
            src_ports.append(0)
            dst_ports.append(0)
            ttls.append(0)
            tcp_flags.append(0)

    #Create a DataFrame from extracted packet data
    packet_df = pd.DataFrame({
        'Timestamp': timestamps,
        'Packet Length': packet_lengths,
        'Direction': directions,
        'srcip': src_ips,
        'dstip': dst_ips,
        'sport': src_ports,
        'dsport': dst_ports,
        'tcp_flags': tcp_flags,
        'ttl': ttls,
    })
    #Filter out packets with unknown direction
    packet_df = packet_df[packet_df['Direction'] != -1]
```

After loading all our data, we then perform calculations and assign values based on our header mapping dictionary and available data (below is a small sample of our calculations, to illustrate how we derive our features):

```
#Derive features from the packet DataFrame
derived = {}
derived['request_id'] = request_id
derived['Sintpkt'] = (packet_df['Timestamp'].diff().mean() * 1_000_000) if len(packet_df) > 1 else 0
derived['smeansz'] = packet_df['Packet Length'].mean()
derived['dmeansz'] = (packet_df.loc[packet_df['Direction'] == 0, 'Packet Length'].mean()
                      if len(packet_df[packet_df['Direction'] == 0]) > 0 else 0)
derived['sloss'] = (packet_df['Direction'].diff() == 0).sum() if len(packet_df) > 1 else 0
derived['dloss'] = ((packet_df['Direction'].diff() == 0) & (packet_df['Direction'] == 0)).sum() if len(packet_df) > 1 else 0

ack_packets = packet_df[packet_df['tcp_flags'].astype(str).str.contains('A')]
if not ack_packets.empty:
    first_a_timestamp = ack_packets['Timestamp'].iloc[0]
    row_timestamp = float(row['Timestamp'])
    derived['ackdat'] = (first_a_timestamp - row_timestamp) * 1_000_000
else:
    derived['ackdat'] = 0
```

As a result of this code, our datasets have been cleaned, merged and any data mismatches have been matched with feature extraction and mathematical derivations.

C.2 Reconstructing Packets into Flows

Following this is our script for matching our packet data to our request data in flows:

```
df_high["protocol"] = pd.to_numeric(df_high["protocol"], errors="coerce").fillna(0).astype(int)
df_high["source port"] = pd.to_numeric(df_high["source port"], errors="coerce").fillna(0).astype(int)
df_high["destination port"] = pd.to_numeric(df_high["destination port"], errors="coerce").fillna(0).astype(int)

#Build a dictionary mapping a key tuple to a unique request identifier (flow id)
#The key is: (source ip, source port, destination ip, destination port, protocol)
#We use this same key reversed to indicate uplink/downlink
request_dict = {}
for idx, row in df_high.iterrows():
    proto_num = row["protocol"]
    if proto_num == 6:
        proto = "tcp"
    elif proto_num == 17:
        proto = "udp"
    else:
        continue

    src_port = str(row["source port"])
    dst_port = str(row["destination port"])
    key = (
        str(row["source ip"]).strip().lower(),
        src_port,
        str(row["destination ip"]).strip().lower(),
        dst_port,
        proto
    )
    req_id = str(row["flow id"]).strip()
    request_dict[key] = req_id
```

First, we construct a tuple for grouping together packets based on flows.

```
with PcapReader(pcap_file) as pcap_reader:
    for pkt in pcap_reader:
        packet_counter += 1

        if IP not in pkt or (TCP not in pkt and UDP not in pkt):
            continue

        src_ip = pkt[IP].src.strip().lower()
        dst_ip = pkt[IP].dst.strip().lower()
        if TCP in pkt:
            proto = "tcp"
            sport = str(pkt[TCP].sport).strip()
            dport = str(pkt[TCP].dport).strip()
        elif UDP in pkt:
            proto = "udp"
            sport = str(pkt[UDP].sport).strip()
            dport = str(pkt[UDP].dport).strip()
        else:
            continue

        #Construct both forward and reverse keys
        key = (src_ip, sport, dst_ip, dport, proto)
        rev_key = (dst_ip, dport, src_ip, sport, proto)

        req_id = request_dict.get(key) or request_dict.get(rev_key)
```

We use the PcapReader function from Scapy to construct packets into our defined tuple format.

D APPENDIX: RNNs

In this appendix, we will provide code examples to illustrate our RNN structure and preprocessing pipeline, as well as any specific differences between our baseline and augmented RNN in terms of data handling.

D.1 Preprocessing Pipeline

Firstly, our tokenizer is initialized as follows:

```
def _create_tokenizer(self):
    pattern = re.compile(r'\b\w+\b|[/=:(),]')
    def tokenize(text):
        text = text.lower()
        text = re.sub(r'0x[0-9a-f]+', ' HEXADDR ', text)
        tokens = pattern.findall(text)
        return [t for t in tokens if t.strip()]
    return tokenize

def _process_info_string(self, info_string):
    tokens = self.tokenizer(info_string)
    embeddings = []
    for token in tokens:
        try:
            embeddings.append(self.embedding_model[token])
        except KeyError:
            embeddings.append(np.zeros(self.embedding_dim))
    return np.array(embeddings)
```

The following code snippets illustrate how we construct our sequential input – the part that loads our GDB data into the RNN. We match request IDs to requests, concatenating sequences of features into an input. We use our tokenizer to properly process packet info and any textual header information:

```
static_features = self.static_features[self.main_df['request_id'] == req_id]
if static_features.shape[0] > 0:
    static_features = static_features[0]
else:
    static_features = np.zeros(self.static_features.shape[1], dtype=np.float32)
if np.isnan(static_features).any():
    static_features = np.nan_to_num(static_features)

packet_df = self.packet_groups.get(req_id, pd.DataFrame())
if packet_df.empty:
    sequence = np.zeros((1, self.sequence_feature_dim), dtype=np.float32)
else:
    time_length = packet_df[["Time", "Length"]].values.astype(np.float32)
    protocols = self.proto_encoder.transform(packet_df[["Protocol"]].values.reshape(-1, 1)).toarray().astype(np.float32)
    info_strings = packet_df['Info'].fillna("").astype(str)
    all_embeddings = [self._process_info_string(info_str) for info_str in info_strings]
    if all_embeddings:
        padded_embeddings = tf.keras.preprocessing.sequence.pad_sequences(
            all_embeddings, dtype='float32', padding='post', truncating='post'
        )
        average_embeddings = np.mean(padded_embeddings, axis=1)
        sequence = np.concatenate([time_length, protocols, average_embeddings], axis=1)
    else:
        sequence = np.concatenate([time_length, protocols], axis=1)
        padding_needed = self.embedding_dim
        padding_array = np.zeros((sequence.shape[0], padding_needed), dtype=np.float32)
        sequence = np.concatenate([sequence, padding_array], axis=1)

        label_str = self.main_df[self.main_df["request_id"] == req_id]['attack_cat'].iloc[0]
        if self.force_known_labels and label_str not in self.class_names:
            label_one_hot = np.zeros(self.n_classes, dtype=np.float32)
        else:
            label_index = self.class_names.index(label_str)
            label_one_hot = np.zeros(self.n_classes, dtype=np.float32)
            label_one_hot[label_index] = 1.0

        batch_static.append(static_features)
        batch_sequences.append(sequence)
        batch_labels.append(label_one_hot)

if not batch_static:
    return (np.empty((0, self.static_features.shape[1]), dtype=np.float32),
            np.empty((0, self.global_max_len, self.sequence_feature_dim), dtype=np.float32)), \
           np.empty((0, self.n_classes), dtype=np.float32)

padded_sequences = tf.keras.preprocessing.sequence.pad_sequences(
    batch_sequences, maxlen=self.global_max_len, dtype='float32', padding='post', truncating='post'
)
return (np.array(batch_static, dtype=np.float32), padded_sequences), np.array(batch_labels, dtype=np.float32)
```

The following changes are applied in our augmented RNN for the added GDB input. Firstly, our tokenizer is utilized differently:

```
# --- Modified _process_info_string ---
def _process_info_string(self, info_string):
    tokens = self.tokenizer(info_string)
    if not tokens:
        return np.zeros((1, self.embedding_dim), dtype=np.float32)
    embeddings = []
    for token in tokens:
        try:
            embeddings.append(self.embedding_model[token])
        except KeyError:
            embeddings.append(np.zeros(self.embedding_dim))
    return np.array(embeddings)
# ------------------------------------
```

So that it can accept hexadecimal values of zero, accounting for some edge cases in our augmented dataset.

Now, let's look at our general data setup and techniques employed in training.

```
novel_classes = [
    'worms',
    'heartbleed',
    'web attack \ufffd sql injection',
    'backdoor',
    'web attack \ufffd brute force',
    'web attack \ufffd xss'
]
known_df = main_df[~main_df['attack_cat'].isin(novel_classes)].copy()
novel_df = main_df[main_df['attack_cat'].isin(novel_classes)].copy()
```

We leave out these minority classes from our training dataset to use as our novel attack cases.

Below is our implementation of SMOTE:

```python
# SMOTE
X_static_list, X_seq_list, y_list = [], [], []
for (static_input, seq_input), labels in train_dataset:
    X_static_list.append(static_input.numpy())
    X_seq_list.append(seq_input.numpy())
    y_list.append(labels.numpy())
X_static = np.concatenate(X_static_list, axis=0)
X_seq = np.concatenate(X_seq_list, axis=0)
y_train = np.concatenate(y_list, axis=0)

sample_fraction = 0.1
num_total = X_static.shape[0]
sample_indices = np.random.choice(num_total, int(sample_fraction * num_total), replace=False)
X_static = X_static[sample_indices]
X_seq = X_seq[sample_indices]
y_train = y_train[sample_indices]

num_samples, global_max_len, seq_feat_dim = X_seq.shape
X_seq_flat = X_seq.reshape(num_samples, global_max_len * seq_feat_dim)
X_combined = np.hstack([X_static, X_seq_flat])
y_train_int = np.argmax(y_train, axis=1)
```

Below is our implementation of Focal Loss:

```python
@tf.keras.utils.register_keras_serializable()
class FocalLoss(tf.keras.losses.Loss):
    def __init__(self, gamma=2.0, alpha=0.25, **kwargs):
        super().__init__(**kwargs)
        self.gamma = gamma
        self.alpha = alpha

    def call(self, y_true, y_pred):
        epsilon = 1e-8
        #Clip predictions to prevent log(0) error
        y_pred = tf.clip_by_value(y_pred, epsilon, 1. - epsilon)
        cross_entropy = -y_true * tf.math.log(y_pred)
        weight = self.alpha * tf.pow(1 - y_pred, self.gamma)
        loss = weight * cross_entropy
        return tf.reduce_mean(tf.reduce_sum(loss, axis=1))

    def get_config(self):
        config = super().get_config()
        config.update({
            "gamma": self.gamma,
            "alpha": self.alpha
        })
        return config
```

D.2 RNN Structure and Hyperparameters

Below is our RNN hyperparameters and structure (no alterations were made for the augmented RNN to ensure the only variable is the input)

```
def build_model(static_feature_dim, sequence_feature_dim, n_classes):
    static_input = Input(shape=(static_feature_dim,))
    x = Dense(32, activation='relu')(static_input)
    x = Dropout(0.3)(x)

    sequence_input = Input(shape=(None, sequence_feature_dim))
    mask = tf.keras.layers.Masking(mask_value=0.0).compute_mask(sequence_input)
    y = GRU(64, return_sequences=True, recurrent_activation='sigmoid')(sequence_input, mask=mask)
    y = Dropout(0.4)(y)
    y = GRU(46, return_sequences=True, recurrent_activation='sigmoid')(y, mask=mask)
    y = Dropout(0.3)(y)

    scores = tf.keras.layers.Dot(axes=(2, 2))([y, y])
    attention_weights = tf.keras.layers.Softmax()(scores)
    context_vector = tf.keras.layers.Dot(axes=(2, 1))([attention_weights, y])
    attention_output = GlobalAveragePooling1D()(context_vector)

    combined = Concatenate()([x, attention_output])
    z = Dense(64, activation='relu')(combined)
    z = Dropout(0.2)(z)
    output = Dense(n_classes, activation='softmax')(z)
    return Model(inputs=[static_input, sequence_input], outputs=output)
```

And then, how our model is constructed using our previously defined class for focal loss, RNN hyperparameters and SMOTE:

```
model = build_model(
    static_feature_dim=train_loader.static_features.shape[1],
    sequence_feature_dim=train_loader.sequence_feature_dim,
    n_classes=n_classes
)

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss=FocalLoss(gamma=2., alpha=0.25),
    metrics=['accuracy',
             tf.keras.metrics.Precision(name='precision'),
             tf.keras.metrics.Recall(name='recall')]
)

model.fit(
    train_dataset_smote,
    epochs=10,
    validation_data=val_dataset
)

model.save("unaugmentedDNN_focal_baseline.keras")
```

E APPENDIX: GDB Commands

This appendix contains the commands we set up our GDB pipeline with; example outputs are located in Chapter 5.3.

```
set logging file [FILENAME].txt
set logging enabled on
break process_packet_sat2
commands
    silent  #Silences GDB output other than our direct commands to it
    finish  #Steps to the end of process execution so we can examine the state of SAT2 post-packet processing
    stepi
    stepi
    stepi
    info locals #Shows local variables after the point of processing
    info args   #Shows what arguments are being passed into the function
    info registers  #Shows processor register states after packet processing
    x/32xb &pinfo->packet   #Shows the memory address buffer after processing a packet
    continue    #Continues execution
    end #Just shows the end of our custom commands

run /tmp/PACKET_DIR #Runs the program with our command setup and with our packet data as input
```

F APPENDIX: C Environment

F.1 Docker and GNS3 VM – Restricted Launch

Our python code is run within a GNS3 VM. First, we have to define a python function to parse python outputs into executable scripts

```
def run_cmd(cmd):
    print("Running:", cmd)
    result = subprocess.run(cmd, shell=True, stdout=subprocess.PIPE, stderr=subprocess.PIPE, text=True)
    if result.stdout:
        print(result.stdout.strip())
    if result.stderr:
        print(result.stderr.strip())
    return result
```

Then, we provide our restricted launch for our docker container:

```
def initialize_containers():
    global SATNET
    create_network(NETWORK_NAME, SUBNET)

    SATNET = run_cmd(f'docker run -d --name {SATNET} --network {NETWORK_NAME} --cap-add=NET_ADMIN',
                     '--privileged --entrypoint /bin/bash gns3/ubuntu:noble -c "tail -f /dev/null"').stdout.strip()
```

Providing compartmentalisation and a safe environment, as described in Chapter 5.3.

Lastly, we initialize our container with modules and our C environment to be run with GDB:

```
run_cmd(f"docker exec {SATNET} apt-get update -y")
run_cmd(f"docker exec {SATNET} apt-get install -y build-essential gdb libpcap-dev gcc tcpdump")

run_cmd(f"docker cp server.c {SATNET}:/tmp/server.c")

run_cmd(f"docker cp /mnt/VM_Shared/pcaps {SATNET}:/tmp/")
```

F.2 C-Based LEO Satellite Environment

In this appendix, we will cover the core functionality and structure of our C environment. Firstly, our packet data structure:

```
// --- Packet Data Structure ---
typedef struct {
    const u_char *packet;  //The actual data of the packet
    int len;
    char src_ip[INET_ADDRSTRLEN];
    char dest_ip[INET_ADDRSTRLEN];
    int protocol;
    int src_port;
    int dest_port;
    char src_mac[18];
    char dst_mac[18];
} PacketInfo;
```

Then, our main packet parser that parses packet data into our defined packet structure:

```
snprintf(pinfo->src_mac, sizeof(pinfo->src_mac), "%02x:%02x:%02x:%02x:%02x:%02x",
         eth_header->ether_shost[0], eth_header->ether_shost[1],
         eth_header->ether_shost[2], eth_header->ether_shost[3],
         eth_header->ether_shost[4], eth_header->ether_shost[5]);
snprintf(pinfo->dst_mac, sizeof(pinfo->dst_mac), "%02x:%02x:%02x:%02x:%02x:%02x",
         eth_header->ether_dhost[0], eth_header->ether_dhost[1],
         eth_header->ether_dhost[2], eth_header->ether_dhost[3],
         eth_header->ether_dhost[4], eth_header->ether_dhost[5]);

if (ntohs(eth_header->ether_type) != ETHERTYPE_IP) return;

struct ip *ip_header = (struct ip *)(packet + sizeof(struct ether_header));
inet_ntop(AF_INET, &(ip_header->ip_src), pinfo->src_ip, INET_ADDRSTRLEN);
inet_ntop(AF_INET, &(ip_header->ip_dst), pinfo->dest_ip, INET_ADDRSTRLEN);

pinfo->protocol = ip_header->ip_p;

if (pinfo->protocol == IPPROTO_TCP) {
    struct tcphdr *tcp_header = (struct tcphdr *)((char *)ip_header + (ip_header->ip_hl * 4));
    pinfo->src_port = ntohs(tcp_header->th_sport);
    pinfo->dest_port = ntohs(tcp_header->th_dport);
} else if (pinfo->protocol == IPPROTO_UDP) {
    struct udphdr *udp_header = (struct udphdr *)((char *)ip_header + (ip_header->ip_hl * 4));
    pinfo->src_port = ntohs(udp_header->uh_sport);
    pinfo->dest_port = ntohs(udp_header->uh_dport);
} else {
    pinfo->src_port = -1;
    pinfo->dest_port = -1;
}
pinfo->packet = packet;
pinfo->len = len;
```

From here, we run packet structures through our topology:

```c
void process_packet(PacketInfo *pinfo, const char *current_hop_mac, bool is_uplink) {
    if (strcmp(current_hop_mac, SAT2_MAC) == 0) {
        process_packet_sat2(pinfo);
        return;
    }

    const char *next_hop = get_next_hop(current_hop_mac, is_uplink);
    if (!next_hop) {
        return;
    }
    simulate_link(current_hop_mac, next_hop, pinfo, is_uplink);
}
```

Using get_next_hop and simulate_link, as described in Chapter 5.2.1:

```c
const char* get_next_hop(const char* current_mac, bool is_uplink) {
    if (is_uplink) {
        if (strcmp(current_mac, UT_MAC) == 0) return SAT1_MAC;
        if (strcmp(current_mac, SAT1_MAC) == 0) return SAT2_MAC;
        if (strcmp(current_mac, SAT2_MAC) == 0) return GS_MAC;
        if (strcmp(current_mac, GS_MAC) == 0) return POP_MAC;
    }
    else {
        if (strcmp(current_mac, POP_MAC) == 0) return GS_MAC;
        if (strcmp(current_mac, GS_MAC) == 0) return SAT2_MAC;
        if (strcmp(current_mac, SAT2_MAC) == 0) return SAT1_MAC;
        if (strcmp(current_mac, SAT1_MAC) == 0) return UT_MAC;
    }
    return NULL;
}
```

```c
void simulate_link(const char *src_mac, const char *dest_mac, PacketInfo *pinfo, bool is_uplink) {
    int delay_us = 0;
    if ((strcmp(src_mac, UT_MAC) == 0 && strcmp(dest_mac, SAT1_MAC) == 0) ||
        (strcmp(src_mac, SAT1_MAC) == 0 && strcmp(dest_mac, UT_MAC) == 0)) {
        delay_us = UPLINK_DELAY_US;
    } else if ((strcmp(src_mac, SAT1_MAC) == 0 && strcmp(dest_mac, SAT2_MAC) == 0) ||
               (strcmp(src_mac, SAT2_MAC) == 0 && strcmp(dest_mac, SAT1_MAC) == 0)) {
        delay_us = ISL_DELAY_US;
    } else if ((strcmp(src_mac, SAT2_MAC) == 0 && strcmp(dest_mac, GS_MAC) == 0) ||
               (strcmp(src_mac, GS_MAC) == 0 && strcmp(dest_mac, SAT2_MAC) == 0)) {
        delay_us = DOWNLINK_DELAY_US;
    }
    usleep(delay_us);
    if (((double)rand() / RAND_MAX) < PACKET_LOSS_PROBABILITY) {
        return;
    }
    process_packet(pinfo, dest_mac, is_uplink);
}
```

As in Chapter 5.2.1, our “nodes” are implicitly defined as follows:

```c
#define UT_MAC   "00:00:00:00:00:01"
#define SAT1_MAC "00:00:00:00:00:02"
#define SAT2_MAC "00:00:00:00:00:03"
#define GS_MAC   "00:00:00:00:00:04"
#define POP_MAC  "00:00:00:00:00:05"
```

These MAC addresses are simply placeholders to define our nodes, they are not used within our network beyond routing, the actual MAC address from each packet is parsed into our data structure in parse_packet.

Lastly, the function we attach GDB to:

```c
void process_packet_sat2(PacketInfo *pinfo) {
    PacketInfo sat2_pinfo;
    parse_packet(pinfo->packet, pinfo->len, &sat2_pinfo);
    bool is_uplink = is_uplink_packet(sat2_pinfo.src_mac);
    const char *next_hop = get_next_hop(SAT2_MAC, is_uplink);
    if (!next_hop) {
        return;
    }
    simulate_link(SAT2_MAC, next_hop, pinfo, is_uplink);
}
```

This function calls separate instances of the standard routing and packet processing function

so that we can ensure that the information we extract with GDB is immediately after the point of packet processing within our satellite, providing us with additional restrictions in our program run for GDB safety.